



SAPIENZA
UNIVERSITÀ DI ROMA

Interazione Uomo Macchina

UNIVERSITÀ DEGLI STUDI DI ROMA "LA
SAPIENZA"

FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE,
INFORMATICA E STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
PROF. MAURIZIO MANCINI

7 dicembre 2024



Prefazione

Salve a tutti!

Mi chiamo Lorenzo Sabatino e sono uno studente del corso di Informatica presso l'Università La Sapienza di Roma. Durante il mio percorso accademico, ho affrontato numerosi esami trascrivendo gli appunti presi a lezione e trasformandoli in veri e propri libri di studio.

Ho voluto condividere questo materiale con voi perché credo che possa essere un valido aiuto per tutti gli studenti di Informatica, sia quelli alle prime armi che chi è già avanti nel percorso di studi. Ho scritto e formattato questi testi con cura, utilizzando il linguaggio di markup $\text{\LaTeX}$ per garantire chiarezza e leggibilità. Ho anche arricchito il contenuto con diagrammi e schemi e ho integrato spunti tratti da lezioni online e da libri di testo.

Tuttavia, è importante sottolineare che questi libri non sostituiscono le lezioni frontali e non sono privi di errori. La mia intenzione è stata quella di fornire un supporto aggiuntivo allo studio, ma è indispensabile partecipare alle lezioni e approfondire i concetti con la guida dei docenti. Inoltre, tenete presente che, data la natura in continua evoluzione dell'informatica, alcune informazioni potrebbero diventare obsolete col passare del tempo.

Nonostante ciò, ho deciso di condividere questi libri perché credo che possano essere d'aiuto a molti studenti e spero che li troviate utili per il vostro percorso accademico.

Se desiderate contattarmi per qualsiasi domanda, commento o correzione, potete scrivere all'indirizzo email scrittinformatica@gmail.com. Sarò lieto di rispondere alle vostre richieste.

Grazie per aver scelto di leggere questi libri e in bocca al lupo per i vostri studi!

Indice

Prefazione	1
1 Introduzione	5
1.1 Interfaccia	5
1.1.1 Storia delle interfacce	5
1.2 Progettazione di sistemi interattivi	6
1.2.1 Il modello di Norman	6
1.2.2 Progettazione centrata sull'utente	7
1.2.3 Usabilità e processo di design centrato sull'uomo	7
2 Needfinding	9
2.1 Metodi di needfinding	9
2.1.1 Osservazione	10
2.1.2 Diari	10
2.1.3 Interviste	11
2.1.4 Sondaggi	12
2.1.5 Inchiesta contestuale	12
2.2 Tecniche principali di needfinding	12
2.2.1 Interviste specifiche	12
2.2.2 Questionari	13
2.2.3 Diario	13
2.2.4 Pager studies	13
2.2.5 Camera studies	13
3 Definire needs, tasks e storyboards	15
3.1 Task analysis	15
3.2 Personas	16
3.3 Storyboards	17
4 Prototyping	21
4.1 Prototipo	21
4.2 Ciclo di prototipazione	22
4.3 Low fidelity prototypes	22
4.3.1 Paper prototypes	22
4.4 Medium fidelity prototypes	23
4.5 High fidelity prototypes	24
4.5.1 Metodo "Wizard of Oz"	25
5 Tecniche di valutazione	27
5.1 Metodi di osservazione	27
5.1.1 Think aloud	28
5.1.2 Cooperative evaluation	28
5.1.3 Protocol analysis	28
5.1.4 Post-task walkthroughs	28
5.2 Esperimenti	28
5.2.1 A/B testing	29
5.3 Basato su esperti	30

5.3.1	Cognitive walkthrough	30
5.3.2	Heuristic evaluation	30
5.3.3	Model-Based evaluation	31
5.3.4	Review-Based evaluation	31
6	Sviluppo di sistemi interattivi	33
6.1	Modello waterfall	33
6.2	Metodo agile	34
6.2.1	Agile UDC	35
7	Interfacce	37
7.1	Modi	37
7.1.1	Errori di modo	38
7.1.2	Noun-verb e verb-noun	38
7.2	Affordances e signifiers	39
8	Design di app mobili	41
8.1	Stili di app mobili	42
8.1.1	Productivity application	42
8.1.2	Utility application	43
8.1.3	Immersive application	43
8.2	Design patterns	44
8.2.1	Dark patterns	45
8.2.2	Attention-capture damaging patterns	45
8.3	Interfacce Android	45
8.3.1	Criteri	46
8.3.2	Material design	47
8.4	Interfacce IOS	49
8.4.1	Criteri	50
8.4.2	Strumenti	50
8.4.3	Human Interface Guidelines	51
	Link utili	53

Capitolo 1

Introduzione

L'interazione uomo-macchina (Human-Computer Interaction o HCI) è una disciplina multidisciplinare che si occupa del design, dell'implementazione e della valutazione di sistemi interattivi destinati all'uso umano. L'obiettivo centrale è progettare sistemi che siano utili, usabili e adottati dagli utenti, concentrandosi sulle loro necessità e sui compiti che devono svolgere.

Il corso si propone di fornire agli studenti una solida comprensione delle teorie, dei modelli e delle regole fondamentali per la progettazione e la validazione di interfacce usabili. L'obiettivo principale è garantire l'acquisizione delle competenze necessarie per progettare sistemi interattivi efficaci, con un forte focus sull'utente finale. Inoltre, verrà introdotta la metodologia Agile, che consente una progettazione iterativa e flessibile, adattandosi ai cambiamenti e ai feedback continui.

In questo contesto, il corso si propone di fornire una comprensione approfondita del processo di progettazione incentrato sull'utente. L'idea è quella di integrare le conoscenze teoriche con l'applicazione pratica, permettendo agli studenti di sviluppare sistemi interattivi che rispondano ai bisogni reali delle persone. Il corso copre anche l'importanza dell'ascolto e della raccolta dei bisogni degli utenti, nonché la valutazione dei sistemi sviluppati con gli utenti stessi.

1.1 Interfaccia

Il concetto fondamentale di interfaccia, visto come il punto d'incontro tra due sistemi indipendenti, in particolare tra l'essere umano e la macchina, ha come obiettivo principale garantire un'efficace comunicazione e controllo della macchina da parte dell'uomo. Le interfacce includono elementi hardware, come i dispositivi fisici, e software, che comprendono i programmi e i sistemi operativi che permettono l'interazione tra l'utente e la macchina.

Le interfacce, quindi, non sono semplici strumenti di input e output, ma costituiscono un vero e proprio ponte tra i due attori: da una parte l'essere umano, con le sue capacità interpretative e creative, e dall'altra la macchina, che segue in modo deterministico le istruzioni ricevute. Questa distinzione tra creatività umana e determinismo meccanico è essenziale per comprendere l'importanza di un'interfaccia efficace, che deve essere in grado di interpretare i comandi umani e restituire risultati utili e comprensibili.

1.1.1 Storia delle interfacce

Un interessante esempio storico della nascita delle interfacce viene dall'abaco, uno dei primi dispositivi di calcolo, utilizzato da civiltà come quella greca e cinese. Qui, l'interfaccia consisteva nel movimento delle dita sugli elementi computazionali, che fornivano un output visibile attraverso la loro disposizione. Con il progredire della tecnologia, le interfacce si sono evolute fino a includere macchine più sofisticate come la Pascalina, costruita da Blaise Pascal nel 1642 per eseguire operazioni aritmetiche, e la macchina di Leibniz per moltiplicazioni e divisioni. Queste interfacce utilizzavano leve e altri meccanismi fisici per manipolare gli elementi computazionali interni.

Un passo importante nella storia delle interfacce è rappresentato dalla macchina analitica di Charles Babbage, considerata il precursore dei computer programmabili. La contessa Ada Lovelace, fi-

glia del poeta Lord Byron, scrisse i primi programmi per questa macchina, gettando le basi per la programmazione moderna.

Le interfacce hanno continuato a evolversi nel corso del XIX secolo con l'introduzione di schede perforate da parte di Hermann Hollerith, utilizzate per il censimento americano del 1890. Queste schede permettevano di codificare informazioni che venivano lette tramite circuiti elettrici, segnando un importante passo avanti verso la digitalizzazione dell'informazione.

Con l'avvento del computer ENIAC, nel 1945, si fece un ulteriore salto tecnologico. L'ENIAC era in grado di eseguire complessi calcoli balistici, e il programma veniva definito collegando circuiti elettrici tramite cavi e interruttori. Questo segnò l'inizio dell'era moderna dei computer, dove l'interfaccia tra uomo e macchina divenne sempre più sofisticata.

Un punto di svolta nella storia delle interfacce si ebbe nel 1984, con la rivoluzione introdotta dalla Apple attraverso la GUI (Graphic User Interface), che utilizzava il paradigma WIMP: Windows, Icons, Mouse, Pointer. Questo nuovo approccio semplificava notevolmente l'interazione con i computer, rendendoli accessibili a un pubblico più vasto.

Negli anni successivi, le interfacce hanno continuato a evolversi, passando dalle classiche interfacce basate su tastiera e mouse a quelle post-WIMP, che includono sistemi di realtà virtuale, interfacce basate su gesti, riconoscimento vocale e persino interfacce tattili e cerebrali. Queste nuove tecnologie mirano a rendere l'interazione con le macchine sempre più naturale e intuitiva.

In conclusione, il passaggio dalle interfacce utente tradizionali all'esperienza utente (UX) rappresenta un cambiamento di paradigma importante: non si tratta più solo di progettare strumenti efficaci, ma di creare esperienze che rispondano ai bisogni e ai desideri degli utenti, rendendo la tecnologia sempre più integrata nella vita quotidiana.

1.2 Progettazione di sistemi interattivi

Progettare un buon sistema interattivo significa seguire un processo iterativo, centrato sugli utenti e sui loro bisogni. Questo processo include diversi passaggi: la raccolta delle necessità, la definizione di principi di design, la creazione rapida di prototipi e la valutazione del sistema. Ogni fase ha l'obiettivo di garantire che il sistema soddisfi gli obiettivi di usabilità, evitando soluzioni che non siano intuitive o che generino errori da parte degli utenti.

L'interazione uomo-macchina, infatti, coinvolge due entità che si influenzano reciprocamente nel tempo: l'uomo e il computer. Per progettare un sistema efficace, bisogna considerare non solo il sistema informatico, ma anche i modelli mentali degli utenti e i loro processi cognitivi.

L'HCI è un campo multidisciplinare, che attinge da diverse aree per garantire che i sistemi siano progettati in modo ottimale. Alcune delle principali discipline coinvolte sono la psicologia, le scienze cognitive e l'ergonomia, che forniscono informazioni sulle capacità fisiche e mentali degli utenti. La sociologia contribuisce a comprendere il contesto sociale in cui avviene l'interazione, mentre l'ingegneria informatica e la grafica aiutano nella costruzione dei sistemi e delle interfacce.

1.2.1 Il modello di Norman

Uno dei modelli più conosciuti nel campo dell'HCI è il modello di interazione di Norman. Questo modello descrive il processo che avviene quando un utente interagisce con un sistema. Il primo passo è la definizione di un obiettivo, che l'utente deve raggiungere attraverso una serie di azioni. Dopo aver eseguito le azioni, l'utente osserva lo stato del sistema per capire se l'obiettivo è stato raggiunto.

L'interazione può essere influenzata da due fattori principali: il *gulf of execution* (il divario tra ciò che l'utente vuole fare e ciò che il sistema permette di fare) e il *gulf of evaluation* (il divario tra ciò che il sistema presenta e ciò che l'utente comprende). La progettazione efficace riduce questi divari, facilitando l'interazione tra l'utente e il sistema.

Uno degli aspetti cruciali della progettazione è la gestione degli errori umani. Secondo Norman, gli errori non dovrebbero mai essere considerati come colpa dell'utente, ma come risultato di una cattiva progettazione. Gli errori umani possono essere suddivisi in due categorie principali: *slip* (quando l'utente sa cosa fare ma esegue l'azione sbagliata) e *mistake* (quando l'utente non comprende bene il sistema e formula un obiettivo errato).

Un buon sistema interattivo dovrebbe anticipare e minimizzare la possibilità di errori, rendendo più facile per l'utente correggere eventuali errori e comprendere lo stato del sistema.

1.2.2 Progettazione centrata sull'utente

La progettazione centrata sull'utente (*User-Centered Design* o *UCD*) pone le necessità, i desideri e le limitazioni degli utenti finali al centro del processo di design. Il coinvolgimento attivo degli utenti durante tutte le fasi del processo di sviluppo assicura che il sistema finale sia più facile da imparare, più efficiente da usare e che riduca il numero di errori umani. Tuttavia, una delle sfide principali è trovare gli utenti giusti e motivati da coinvolgere nel processo, e tradurre le loro esigenze in requisiti di design.

Il design partecipativo va oltre l'UCD, coinvolgendo direttamente gli utenti nel processo di progettazione attraverso la creazione di scenari, schizzi e prototipi. Questo approccio è particolarmente utile quando si progetta per utenti esperti o popolazioni mature, in quanto richiede una collaborazione attiva e costante da parte degli utenti coinvolti.

Il design thinking è un approccio incentrato sull'innovazione, che combina le esigenze degli utenti, le possibilità tecnologiche e i requisiti di successo del business. Il processo non lineare e iterativo del design thinking prevede cinque fasi: empatia, definizione dei bisogni, ideazione, prototipazione e test.

D'altro canto, la progettazione agile è un metodo di sviluppo evolutivo che si basa su cicli di rilascio rapidi e iterazioni continue. Questo approccio favorisce la creazione di prototipi a basso costo e richiede ispezioni rapide dell'usabilità, cercando di ridurre al minimo i tempi di sviluppo.

Il design dei servizi rappresenta un cambiamento contemporaneo verso la progettazione di esperienze complete, piuttosto che di prodotti isolati. Un esempio tipico è quello del passaggio dalla vendita di automobili come prodotti, all'offerta di servizi di mobilità come Uber. Questo tipo di design si basa su cinque principi fondamentali: centralità dell'utente, co-creazione, sequenzialità, evidenziazione e approccio olistico.

1.2.3 Usabilità e processo di design centrato sull'uomo

Infine, l'usabilità è un concetto centrale nell'HCI. Definita dall'ISO come

"la misura in cui un sistema può essere usato da utenti specifici per raggiungere determinati obiettivi in modo efficace, efficiente e soddisfacente"

L'usabilità include diverse dimensioni: utilità, facilità di apprendimento, memorizzabilità, efficacia, efficienza, visibilità dello stato del sistema, gestione degli errori e soddisfazione dell'utente.

Il processo di progettazione centrato sull'utente si sviluppa attraverso diverse fasi: il needfinding (ovvero la raccolta dei bisogni), l'analisi, il design, l'iterazione e prototipazione, e infine l'implementazione e il dispiegamento del sistema.

Capitolo 2

Needfinding

Il termine **needfinding** si riferisce all'attività di identificare le necessità potenziali degli utenti e le lacune nei sistemi attuali, per trovare opportunità di miglioramento. E' importante non solo riconoscere i problemi esistenti, ma anche identificare come questi possono trasformarsi in opportunità per creare soluzioni migliori.

Nel contesto del design centrato sull'utente (Human-Centered Design), il needfinding è uno dei primi passi essenziali. Si tratta di un processo esplorativo, dove i progettisti cercano di scoprire i bisogni latenti degli utenti, osservando attentamente i loro comportamenti e le interazioni con i sistemi attuali. Non ci si limita a chiedere agli utenti cosa desiderano, ma si esplora il contesto in cui operano e come utilizzano i prodotti esistenti.

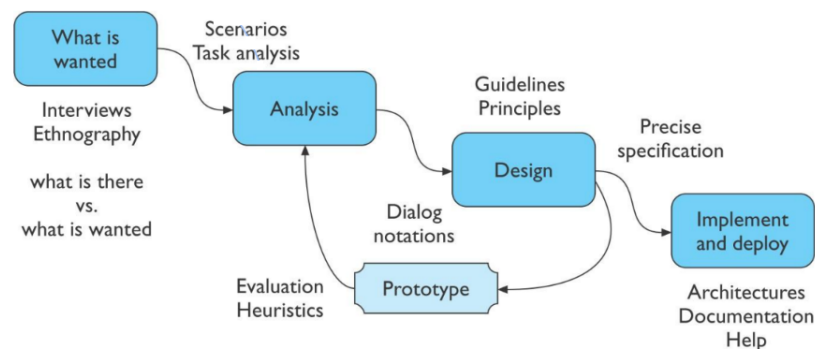


Figura 2.1: Human-Centered Design Process

Il needfinding si concentra nel riconoscere i "vuoti" o gap all'interno di un sistema. Un esempio pratico di questo concetto è rappresentato dalla ricerca su come le persone utilizzano un'applicazione di benessere digitale. Attraverso l'osservazione, possiamo identificare problemi che magari gli utenti non esprimono apertamente, come difficoltà nell'uso o comportamenti di adattamento (*workaround*). Questo processo permette ai progettisti di rispondere a domande fondamentali come: *Chi sono gli utenti? Come stanno utilizzando il sistema ora? In quale contesto lo usano?*

Per condurre un'efficace attività di needfinding, è cruciale identificare correttamente i veri utenti. Un errore comune è presumere che i progettisti stessi o i clienti del progetto siano utenti rappresentativi, ma in realtà le loro competenze e conoscenze sono molto diverse. Bisogna invece parlare con le persone che effettivamente useranno il sistema.

2.1 Metodi di needfinding

Uno dei metodi più utilizzati per raccogliere informazioni è parlare direttamente con gli utenti. Si possono usare tecniche come interviste, questionari o coinvolgimento diretto nel processo di design. E' importante comprendere il comportamento reale degli utenti, ascoltando attentamente i loro problemi e cercando di cogliere le soluzioni che adottano per superare le difficoltà (*workaround*).

2.1.1 Osservazione

L'*osservazione* è uno strumento fondamentale. Si tratta di osservare il comportamento degli utenti in modo naturale, senza interferenze. Un classico esempio è lo studio etnografico, che consiste nell'immergersi nel contesto dell'utente per capire meglio come funziona l'ambiente in cui opera. L'obiettivo è raccogliere dati che influenzeranno il redesign dell'interfaccia, imparando il "linguaggio" dell'utente e del suo ambiente.

Un altro punto importante che emerge riguarda la differenza tra processo e pratica. Il processo rappresenta il modo in cui le cose dovrebbero essere fatte ufficialmente, secondo la formazione, mentre la pratica riflette i trucchi appresi sul campo. Ad esempio, un impiegato potrebbe avere un metodo ufficiale per completare una determinata operazione, ma nel corso del tempo ha sviluppato delle scorciatoie o soluzioni personali che lo aiutano a lavorare più velocemente.

Esistono due principali tipi di osservazione: quella controllata in un ambiente di laboratorio e quella naturalistica sul campo. La prima è più facile da riprodurre e analizzare, poiché i dati sono spesso quantitativi e replicabili. Tuttavia, la seconda offre un contesto più realistico, permettendo agli utenti di affrontare le reali difficoltà che incontrano nella vita quotidiana. Ad esempio, l'osservazione di una persona che utilizza uno smartphone in condizioni di pioggia potrebbe rivelare sfide che non emergerebbero in laboratorio. La raccolta dei dati può essere di due tipi: soggettiva (basata su impressioni, valutazioni e racconti degli utenti) o oggettiva (come i dati registrati automaticamente da un sistema). Entrambi i tipi sono utili per costruire un quadro completo delle necessità degli utenti.

Tra i rischi dell'osservazione vi è il cosiddetto effetto *hawthorne*, dove la semplice presenza dell'osservatore modifica il comportamento dell'utente. Per questo, è fondamentale adottare tecniche che riducano l'impatto dell'osservatore, come il *blending in* (letteralmente, "confondersi con l'ambiente").

Come ha ben sintetizzato Uri Levine, fondatore di Waze: "*Innamorati del problema, non della soluzione*". Questa frase evidenzia quanto sia essenziale comprendere il vero problema, piuttosto che innamorarsi di una soluzione predefinita. L'osservazione, infatti, consente di definire con maggiore precisione il problema, e permette di scoprirlo in modo autentico e senza preconcetti. L'osservazione, infatti, deve essere condotta con una mente aperta, senza cercare conferme per idee o soluzioni a priori. Bisogna essere pronti a scoprire dettagli imprevisti e lasciarsi sorprendere. Per questo motivo, l'osservatore deve prestare attenzione non solo a cosa fanno gli utenti, ma anche a quali sono i loro obiettivi, che strumenti utilizzano, il contesto in cui operano e le attività più ampie di cui fanno parte. Quando si osservano gli utenti, è fondamentale non limitarsi a ciò che stanno facendo in quel preciso momento. Bisogna cercare di capire il loro obiettivo finale e come esso si collega all'attività specifica. E' altrettanto importante considerare il contesto in cui si trovano, inclusi fattori come il momento della giornata, l'ambiente fisico e le differenze tra gli utenti stessi. Dettagli come questi possono rivelare problematiche nascoste o opportunità di miglioramento.

Un metodo classico di osservazione è quello di passare del tempo vicino all'utente senza disturbarlo, lasciandolo agire naturalmente. Questo può essere fatto in luoghi pubblici come strade, locali o mezzi di trasporto, oppure attraverso l'analisi di video registrati, che consente uno studio più approfondito del comportamento senza interferenze dirette. Un altro approccio importante è quello di "mettersi nei panni" dell'utente, partecipando attivamente alle loro attività. In questo modo, si possono scoprire dettagli che sfuggirebbero a un'osservazione passiva. Bisogna prestare particolare attenzione a eventuali trucchi, scorciatoie o errori commessi dagli utenti. Lavorare insieme a loro permette di ottenere una comprensione profonda dei loro reali bisogni. Ascoltare gli utenti è tanto importante quanto osservarli. Spesso, ciò che gli utenti esprimono verbalmente non coincide esattamente con le loro azioni. Gli osservatori devono evitare di influenzare le risposte degli utenti con domande dirette o intrusive e concentrarsi invece su emozioni, paure e frustrazioni. Questo approccio consente di ottenere una visione più autentica e genuina dei bisogni degli utenti.

E' fondamentale non confondere i need (bisogni) con le soluzioni. I bisogni sono la chiave per aprire nuove opportunità, mentre le soluzioni possono limitare l'innovazione. Partire con una soluzione già in mente rischia di impedire l'immaginazione di qualcosa di completamente nuovo o rivoluzionario. La pazienza è cruciale, perché anche se le osservazioni possono sembrare inizialmente banali, è proprio nei piccoli dettagli che si possono nascondere intuizioni preziose.

2.1.2 Diari

Il metodo dei diari è utile quando non è possibile osservare direttamente gli utenti per lunghi periodi di tempo. In questo caso, si richiede agli utenti di tenere traccia delle loro attività quotidiane,

annotando le azioni rilevanti. I diari possono essere cartacei o digitali, e gli utenti registrano le loro azioni quando eseguono specifiche operazioni o a intervalli di tempo prestabiliti.

L'uso dei diari consente di raccogliere dati su un lungo periodo di osservazione, fornendo informazioni dettagliate che potrebbero sfuggire in una semplice sessione di osservazione. Tuttavia, mantenere la motivazione degli utenti è essenziale: potrebbe essere utile offrire incentivi per assicurarsi che completino correttamente il diario.

2.1.3 Interviste

Le interviste sono un altro metodo importante per raccogliere dati dagli utenti. Si può scegliere tra interviste strutturate (con domande predeterminate) e non strutturate (che lasciano maggiore libertà). Le interviste possono essere condotte faccia a faccia o in gruppo (focus group). Rispetto ai sondaggi, le interviste permettono di ottenere conoscenze più profonde e dettagliate, ma richiedono molto tempo.

Un rischio delle interviste è che gli utenti potrebbero non sapere cosa vogliono realmente o potrebbero cercare di rispondere ciò che pensano l'intervistatore voglia sentire. Inoltre, per prodotti o tecnologie completamente nuove, gli utenti potrebbero non avere la creatività necessaria per immaginare potenziali applicazioni. Infatti, gli utenti spesso non riescono a esprimere esplicitamente i loro bisogni. Ad esempio, prima dell'invenzione dell'iPhone, nessuno avrebbe chiesto un telefono touch-screen con app, perché nessuno poteva immaginare quel prodotto. Questo evidenzia l'importanza di andare oltre le semplici richieste degli utenti e investigare i loro bisogni più profondi.

Nella scelta dei partecipanti alle interviste, è essenziale includere rappresentanti del target di utenti. In alcuni casi, può essere utile coinvolgere utenti attuali di sistemi simili, oppure anche i non utenti, se si sta progettando un prodotto completamente nuovo. È importante scegliere con cura i partecipanti e, quando possibile, offrire piccole ricompense o incentivi per la loro partecipazione.

Per condurre interviste efficaci, bisogna creare un ambiente confortevole per l'utente, introdurre chiaramente l'obiettivo dell'intervista e spiegare che non si sta valutando la persona, ma si vuole ottenere il suo aiuto. Durante l'intervista, è cruciale dare tempo sufficiente agli utenti per rispondere e seguire le loro risposte con domande di approfondimento.

Nella progettazione delle domande, bisogna distinguere tra domande strutturate e non strutturate. Le prime sono più facili da analizzare, mentre le seconde permettono risposte più libere e dettagliate. È sempre utile includere domande aperte per esplorare meglio le opinioni e le esperienze degli utenti, ma anche chiedere spiegazioni ulteriori quando gli utenti forniscono valutazioni numeriche (ad esempio, chiedendo cosa intendono con un punteggio "4" su una scala da 1 a 5).

Alcune domande rischiano di introdurre bias o influenzare negativamente le risposte degli utenti. Ad esempio, domande come *"Quanto è importante per te la funzione X?"* possono indurre gli utenti a rispondere positivamente, anche se in realtà non considerano la funzione rilevante. Altre domande che richiedono risposte ipotetiche o chiedono di immaginare scenari futuri tendono a generare risposte poco affidabili.

Per trovare partecipanti alle interviste, è possibile utilizzare i social network, la rete di contatti personali o incentivare le persone a partecipare. Anche se non sempre si riesce a ottenere il partecipante ideale, è comunque possibile raccogliere informazioni utili, poiché tutti hanno qualcosa di interessante da dire.

Quando si prepara un'intervista, è fondamentale avere un piano ben strutturato. Le domande devono essere preparate in anticipo, la privacy deve essere rispettata ottenendo il consenso per la registrazione, e l'intervistato deve essere incoraggiato a espandere le risposte. Un buon intervistatore deve mantenersi neutrale, evitando di influenzare l'intervistato con giudizi personali. Ci sono alcuni tipi di domande che è meglio evitare, come quelle la cui risposta è scontata o quelle che contengono già la risposta. Domande troppo generiche o basate su scenari ipotetici possono portare a risposte poco utili. È invece preferibile porre domande aperte e concrete, focalizzandosi su situazioni reali e ben conosciute dall'intervistato. Le interviste dovrebbero seguire una struttura precisa. Si inizia con un'introduzione per spiegare gli scopi dell'intervista e ottenere il consenso. Poi, si procede con domande più semplici e demografiche per mettere a proprio agio l'intervistato, passando successivamente alle domande più profonde. Infine, si conclude con una fase di cooling off, con domande facili per allentare la tensione e ringraziare l'intervistato.

2.1.4 Sondaggi

I sondaggi sono strumenti familiari e poco costosi per raccogliere dati da un ampio pubblico. Tuttavia, essi tendono a fornire una visione più superficiale rispetto a metodi come le interviste o le osservazioni. E' fondamentale definire chiaramente l'obiettivo del sondaggio prima di progettare le domande, in modo da raccogliere dati rilevanti.

Tra i rischi associati ai sondaggi c'è il fatto che si basano spesso sulla memoria degli utenti, che può essere inaffidabile. Inoltre, i sondaggi non consentono domande di approfondimento e possono generare dati distorti se trattano temi sensibili come denaro o emozioni.

Un buon sondaggio deve avere sezioni mirate, a partire da una spiegazione chiara dello scopo e del tempo necessario per completarlo. E' utile chiedere informazioni di background sugli utenti, come età, livello di istruzione e familiarità con il sistema, limitando al minimo i campi obbligatori per non scoraggiare la partecipazione.

Esistono vari tipi di domande da includere in un sondaggio, come quelle aperte, che richiedono risposte dettagliate, e quelle chiuse, dove l'utente deve scegliere tra opzioni predefinite. Le domande chiuse possono avere scale ordinarie (ad esempio, valutazioni su una scala da 1 a 5) o nominali (scelte senza un ordine preciso, come città di residenza).

La scala Likert è uno degli strumenti più utilizzati nei sondaggi per misurare l'accordo o il disaccordo degli utenti su determinate affermazioni. Nelle versioni a 4 o 5 livelli, gli utenti valutano affermazioni su una scala che va da "fortemente in disaccordo" a "fortemente d'accordo".

2.1.5 Inchiesta contestuale

L'inchiesta contestuale combina l'osservazione con l'intervista, permettendo di osservare gli utenti nel loro ambiente naturale mentre descrivono ciò che stanno facendo. Questo approccio è particolarmente utile per comprendere il ragionamento degli utenti dietro determinate azioni e per rilevare problemi che potrebbero non emergere in un'intervista tradizionale.

Uno dei principali rischi è che gli utenti potrebbero entrare in modalità "intervista" durante l'osservazione, cercando di riassumere i loro processi o di evidenziare solo i problemi del sistema. E' importante che l'intervistatore mantenga un approccio obiettivo e aperto, senza pregiudizi.

2.2 Tecniche principali di needfinding

2.2.1 Interviste specifiche

Le interviste sono uno degli strumenti più comuni nel needfinding. Tra i tipi di interviste specifiche, quelle con i *lead users* si rivelano particolarmente efficaci per identificare tendenze future. I *lead users* sono utenti avanzati che sperimentano bisogni nuovi prima di chiunque altro e trovano soluzioni ai propri problemi, anticipando le necessità che potrebbero manifestarsi per una popolazione più ampia in futuro. Grazie alla loro competenza, questi utenti possono fornire suggerimenti su come migliorare il sistema in base a esperienze avanzate. Un esempio pratico potrebbe riguardare un programmatore che crea script personalizzati per automatizzare attività che altri utenti non hanno ancora percepito come problematiche.

Gli *extreme users*, d'altra parte, spingono i sistemi ai loro limiti, rivelando problemi e necessità che altrimenti sarebbero difficili da individuare. Gli utenti estremi forniscono visioni su necessità condivise con altri utenti, ma che risultano più visibili a causa delle loro esigenze amplificate.

Le *interviste con esperti* si concentrano su professionisti che hanno una conoscenza approfondita del settore o del prodotto. Questi esperti possono fornire informazioni aggregate e generalizzate sui comportamenti degli utenti e discutere di problemi astratti o tecnici che non sempre emergono dalle interviste agli utenti finali. Ad esempio, un ingegnere esperto di interfacce potrebbe spiegare come certi limiti tecnici influenzano negativamente l'esperienza d'uso.

Oltre alle interviste classiche, esistono altri formati utili per raccogliere informazioni:

- *History interviews*: comprendono la storia e le sequenze di eventi che spiegano i comportamenti degli utenti.
- *Process mapping*: richiedono all'intervistato di descrivere l'intero processo per compiere un'azione.

- *Laddering*: una tecnica basata sulla ripetizione della domanda "Perché?" per approfondire i motivi alla base di un comportamento.
- *Cultural context*: interviste non strutturate per comprendere il contesto culturale dell'utente.
- *Intercepts*: rapide interviste che pongono una singola domanda per raccogliere dati immediati.

2.2.2 Questionari

I questionari rappresentano un metodo efficiente per raccogliere informazioni da un ampio numero di utenti in poco tempo. Si tratta di un insieme di domande uguali per tutti, che può essere analizzato in modo rigoroso. I questionari possono includere domande aperte, chiuse, con risposta singola, multipla, o basate su scale di valori. Tuttavia, rispetto alle interviste, i questionari sono meno flessibili. Per essere efficaci, devono essere progettati accuratamente, con particolare attenzione al modo in cui le risposte verranno analizzate.

2.2.3 Diario

Il metodo del diario si utilizza per raccogliere informazioni su attività che si verificano sporadicamente o per studi longitudinali. Ad esempio, si potrebbe chiedere agli utenti di tenere traccia delle loro interazioni con un software di gestione del tempo nel corso di un mese, annotando i momenti in cui lo usano e le difficoltà incontrate. Esistono anche applicazioni digitali che facilitano la registrazione delle informazioni in tempo reale.

2.2.4 Pager studies

I pager studies chiedono agli utenti di interrompere le loro attività quotidiane in determinati momenti per annotare le loro esperienze. Questo metodo è utile per catturare dati specifici in tempo reale, sia in relazione a un particolare momento della giornata sia a un particolare luogo. Gli utenti possono prendere note libere o compilare moduli prestabiliti. I dati raccolti servono a identificare pattern o ripetizioni nel comportamento.

2.2.5 Camera studies

Negli studi con videocamere, agli utenti viene fornita una videocamera per riprendere le loro attività. Questo approccio permette ai progettisti di vedere le interazioni dal punto di vista dell'utente, ottenendo una visione più immersiva e dettagliata di come utilizzano un prodotto o un servizio.

Capitolo 3

Definire needs, tasks e storyboards

3.1 Task analysis

Il Task Analysis è un metodo di analisi usato per comprendere come gli utenti eseguono determinate attività, con l'obiettivo di progettare sistemi interattivi che rispondano efficacemente ai loro bisogni. Questo processo consente di raccogliere informazioni su ciò che gli utenti fanno, quali strumenti utilizzano e come raggiungono i loro obiettivi, aiutando i progettisti a sviluppare interfacce più intuitive e mirate.

Nel processo di progettazione centrata sull'utente, il Task Analysis è un passaggio intermedio tra l'analisi dei bisogni dell'utente e il design dell'interfaccia. Serve a rendere espliciti i bisogni degli utenti e a rappresentare i risultati dell'analisi in modo chiaro. Nel processo, i bisogni vengono tradotti in obiettivi concreti, che a loro volta si trasformano in azioni. Un esempio pratico potrebbe essere il bisogno di "migliorare il benessere", che si traduce nell'obiettivo di "fare esercizio fisico regolarmente" o "monitorare il sonno", e infine nelle azioni di utilizzare app per il monitoraggio delle attività fisiche o del sonno. Tuttavia, il contesto influisce sugli obiettivi, ma non sui bisogni di fondo. Ad esempio, il bisogno di "sentirsi sicuri" può tradursi in un obiettivo come "ottenere una promozione" o "installare un sistema di sicurezza a casa", a seconda del contesto.

Il Task Analysis è lo studio di come le persone svolgono le loro attività quotidiane. L'obiettivo principale è determinare:

1. *Cosa fanno*: quali sono i passaggi necessari per completare un'attività.
2. *Quali strumenti usano*: gli artefatti che impiegano per svolgere le attività.
3. *Quanto bene riescono a completare l'attività*: quali sono i loro obiettivi e quanto successo hanno nel raggiungerli.

Secondo Benyon, *"un task è definito come un obiettivo accompagnato da una serie ordinata di azioni necessarie per raggiungerlo"*. Esistono diversi livelli di astrazione per descrivere un task:

- *Goal*: lo stato che l'utente vuole raggiungere.
- *Task*: un insieme strutturato di attività necessarie per raggiungere il goal usando una specifica tecnologia.
- *Action*: un task semplice che non richiede risoluzione di problemi o controllo.

Attraverso l'analisi dei task, si ottengono informazioni dettagliate su cosa gli utenti cercano di ottenere, come eseguono i task e quali esperienze personali, sociali e culturali influenzano il loro lavoro. Si apprende anche come l'ambiente fisico e l'esperienza pregressa influenzino il workflow degli utenti e i punti di difficoltà che incontrano.

Tuttavia, una delle principali sfide è che i progettisti non creano i task, bensì le interfacce. Questo significa che i task e gli oggetti non hanno sempre una corrispondenza uno-a-uno. Ad esempio, un'applicazione web può supportare molteplici task diversi, e le persone possono usare la stessa interfaccia per ottenere risultati leggermente differenti o per eseguire attività diverse.

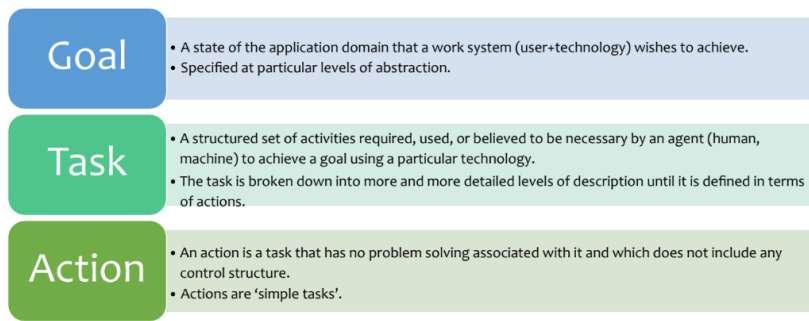


Figura 3.1: Goal, task e action

Esempio 3.1.1 – Uber

Esaminiamo un servizio come Uber.

Gli utenti di Uber possono avere diversi bisogni:

1. *Necessità di un trasporto rapido o sicuro:* l'utente desidera un modo rapido e semplice per andare dal punto A al punto B senza la seccatura di cercare un mezzo pubblico o di guidare da solo.

Un task associato al bisogno di trasporto conveniente potrebbe essere quello di "richiedere un'auto attraverso l'app". In questo caso, l'interfaccia permette all'utente di scegliere un punto di partenza e una destinazione, trovando un autista disponibile nelle vicinanze.

2. *Necessità di sicurezza durante il viaggio:* l'utente desidera sentirsi al sicuro durante il viaggio, sia tramite conducenti controllati, monitoraggio in tempo reale o assicurandosi che qualcuno conosca la sua posizione.

Il task associato potrebbe consistere nel "verificare i dettagli del conducente e monitorare il progresso del viaggio". L'app fornisce informazioni sull'identità e i dettagli del veicolo del conducente, e consente all'utente di tracciare la posizione del veicolo in tempo reale.

3. *Necessità di trasporto conveniente:* l'utente desidera bilanciare la comodità con l'accessibilità economica e assicurarsi di ottenere un viaggio a un prezzo ragionevole.

Il task sarebbe "valutare le opzioni di trasporto in base al costo e alla disponibilità". L'app permette di confrontare diverse opzioni (come corse condivise o private) e fornisce stime di costo, consentendo all'utente di scegliere l'opzione che meglio si adatta al suo budget.

3.2 Personas

Nel design dell'Interazione Uomo-Macchina (HCI), uno degli strumenti più utili per migliorare la progettazione delle interfacce è il concetto di *personas*. Le *personas* rappresentano modelli astratti di utenti reali, basati su dati concreti, e vengono create per comprendere meglio le esigenze e i comportamenti degli utenti durante il processo di design.

Una persona non è semplicemente una caricatura di un utente generico, ma una rappresentazione semi-fittizia che si basa su dati demografici, comportamentali e psicografici reali. Queste informazioni permettono ai progettisti di modellare i diversi tipi di utenti con cui dovranno interagire. L'obiettivo è quello di suddividere gli utenti in categorie con caratteristiche, obiettivi e necessità specifiche, creando un'immagine dettagliata che diventi un riferimento per tutto il team di progettazione.

Le *personas* aiutano i progettisti a mettersi nei panni degli utenti reali, creando empatia. Immaginare un utente concreto con desideri, bisogni e frustrazioni, permette ai designer di comprendere meglio il punto di vista degli utenti. Questo processo facilita anche il miglioramento dell'esperienza utente (UX), in quanto le *personas* diventano uno strumento guida per le decisioni di design. Quando i progettisti si trovano a dover scegliere tra diverse soluzioni, possono chiedersi cosa preferirebbe

una determinata persona, aiutando a ottimizzare il prodotto per il target specifico. Le personas forniscono anche un quadro di riferimento che aiuta a identificare esigenze e sfide degli utenti, creando così un focus più preciso nella progettazione. Questo è particolarmente utile quando ci si rende conto che non è possibile progettare un'interfaccia che vada bene per tutti, poiché si rischierebbe di non soddisfare pienamente nessuno.

Ogni persona è composta da diversi elementi chiave che forniscono una rappresentazione completa dell'utente:

1. *Nome*: Dare un nome alla persona rende l'utente più umano e più facile da visualizzare per il team di progettazione.
2. *Foto*: Un'immagine rafforza l'identità della persona, rendendo l'utente fittizio più reale e favorendo un maggiore coinvolgimento emotivo da parte dei progettisti.
3. *Caratteristiche demografiche*: Età, genere, professione e livello di istruzione sono tutti dati essenziali per comprendere meglio il background dell'utente e il modo in cui potrebbe interagire con il sistema.
4. *Obiettivi*: Cosa vuole ottenere l'utente utilizzando l'interfaccia? Ad esempio, potrebbe voler completare rapidamente un'azione o potrebbe cercare di esplorare nuove funzionalità.
5. *Frustrazioni*: Quali sono i principali problemi che l'utente potrebbe incontrare durante l'uso del sistema? Questi possono includere difficoltà tecniche o sfide specifiche legate alla comprensione dell'interfaccia.
6. *Comportamenti*: Le abitudini e il modo in cui l'utente interagisce con il sistema sono cruciali per progettare soluzioni che siano intuitive e in linea con le aspettative.

L'uso delle personas porta numerosi vantaggi nel processo di progettazione delle interfacce. Uno dei benefici principali è che aiutano i designer a creare empatia con l'utente. Interiorizzando gli obiettivi e i bisogni dell'utente fittizio, i progettisti possono prendere decisioni più accurate, basate su una visione concreta di chi utilizzerà il prodotto. Un altro vantaggio è la capacità di sviluppare un focus: le personas rendono chiaro che non è possibile progettare per tutti. Concentrarsi su utenti specifici permette di evitare un design troppo generico e privo di personalità. Inoltre, le personas aiutano a comunicare le conclusioni della ricerca, condividendo intuizioni e risultati con coloro che non hanno partecipato direttamente agli incontri con gli utenti, come stakeholder e altri membri del team. Le personas facilitano anche il processo decisionale, in quanto permettono di vedere il mondo dalla prospettiva dell'utente, rendendo più semplice determinare quali funzionalità sono utili e quali potrebbero essere casi limite. Inoltre, possono fungere da sostituti degli utenti durante la fase di test, aiutando a misurare l'efficacia delle soluzioni proposte.

3.3 Storyboards

Nel campo dell'Interazione Uomo-Macchina (HCI), gli *storyboards* rappresentano un potente strumento di comunicazione visiva utilizzato per raffigurare l'aspetto esteriore di un sistema, senza mostrare le sue funzionalità interne. Sono spesso utilizzati come supporto per illustrare come si svolgono i task da parte degli utenti, il contesto in cui l'utente utilizzerà il sistema, i passaggi principali del task e l'interazione tra gli utenti e l'ambiente. Vengono rappresentati in forma di fumetti disegnati a mano, con una sequenza di sketch che rappresentano i passaggi chiave di un'azione.

Uno storyboard è una rappresentazione grafica che mostra come una persona può portare a termine un task attraverso l'interazione con un sistema o un'interfaccia. La sua natura "fumettistica" rende il processo immediato, permettendo ai progettisti di concentrarsi sul flusso del task, piuttosto che sui dettagli tecnici o visivi dell'interfaccia. Uno degli elementi distintivi degli storyboard è la presenza di persone: infatti, uno storyboard efficace deve sempre includere individui che interagiscono con l'interfaccia o tra di loro.

Uno degli aspetti più utili degli storyboard è la loro capacità di rappresentare cosa accade nei momenti chiave di un task, mostrando come un utente interagisce con il sistema in diverse fasi temporali. Non è richiesta alcuna particolare abilità artistica, poiché lo scopo principale non è creare immagini "belle", ma comunicare le idee. I progettisti devono quindi concentrarsi sul raccontare una storia e far emergere il contesto in cui si svolge l'interazione.



Figura 3.2: Storyboards

Uno storyboard efficace deve includere diversi aspetti fondamentali. Il primo è l'illustrazione di un obiettivo, ovvero cosa l'utente cerca di ottenere con il task rappresentato. Successivamente, deve mostrare come il task si sviluppa, cioè come l'utente interagisce con il sistema e con altre persone o dispositivi. Ogni passo significativo del task deve essere rappresentato in modo chiaro, fino a mostrare l'esito finale, che illustra come l'utente raggiunge il proprio obiettivo in modo soddisfacente.

È importante sottolineare che in questa fase non si dovrebbero ancora disegnare le schermate dell'interfaccia. Rappresentare già le schermate potrebbe vincolare troppo le decisioni sul design, mentre in questo momento l'obiettivo è comprendere come si sviluppano i task e come gli utenti interagiscono con il sistema.

Esempio 3.3.1 – Storyboard - Ricetta

Immaginiamo uno storyboard che mostra una persona che utilizza un'app cercare una ricetta da preparare per cena.

Le prime immagini illustrano l'utente che finisce di lavorare, prende la metropolitana e prova a utilizzare l'app ma c'è assenza di segnale. Successivamente consulta la lista della spesa per acquistare gli ingredienti e, infine, prepara il pasto una volta a casa.



Lo storyboard ci permette di capire che l'utente vorrebbe usare l'applicazione anche quando c'è assenza di segnale. Inoltre riusciamo ad intuire che l'app dovrebbe consigliare una ricetta, mostrare le calorie, gli ingredienti e le istruzioni per prepararla.

Esempio 3.3.2 – Storyboard - Cucina

Questo storyboard mostra come un'app può convincere un utente che cucinare un pasto a casa sia più veloce rispetto a ordinare del cibo da asporto.

In questo caso, lo storyboard rappresenta l'interfaccia dell'app mentre mostra ricette con ingredienti che l'utente ha già nel frigorifero, rendendo la preparazione del pasto immediata.



Ogni storyboard dovrebbe riuscire a comunicare tre elementi principali: *ambientazione*, *sequenza* e *soddisfazione*. L'ambientazione include i personaggi, ovvero le persone coinvolte, l'ambiente in cui si trovano e il compito che stanno svolgendo. La sequenza mostra i passaggi necessari per portare a termine il task, senza però soffermarsi sui dettagli tecnici dell'interfaccia utente. Il ruolo dell'interfaccia deve essere evidenziato come uno strumento che facilita il raggiungimento dell'obiettivo dell'utente. Infine, lo storyboard deve trasmettere il concetto di soddisfazione, mostrando cosa motiva l'utente a completare il task e qual è il risultato finale che desidera ottenere. Questa parte è fondamentale, poiché sottolinea come il sistema o l'app rispondano a un bisogno reale dell'utente.

Esistono diversi tipi di storyboard, ognuno adatto a situazioni specifiche. Lo storyboard tradizionale segue convenzioni simili a quelle dei fumetti, con personaggi, nuvolette di dialogo e un sfondo che illustra l'ambiente. Spesso si aggiungono note a ogni scena per spiegare ciò che sta accadendo. Quando l'interfaccia è molto dinamica o contiene elementi multimediali specifici, si possono usare scored storyboards, che includono annotazioni aggiuntive su movimenti, colori e suoni. Infine, nei casi in cui l'interazione è troppo complessa per essere rappresentata visivamente, si possono utilizzare storyboard testuali, che descrivono in modo dettagliato il comportamento dell'utente e del sistema.

Perché disegnare a mano? Disegnare uno storyboard a mano ha diversi vantaggi. È un metodo rapido, che non richiede tempo da dedicare a strumenti di grafica, i quali spesso spingono i progettisti a focalizzarsi troppo sui dettagli estetici piuttosto che sul flusso del task. La natura imprecisa del disegno a mano fa sì che gli utenti si sentano più liberi di esprimere commenti e suggerimenti, rispetto a versioni più "raffinate" o professionali, che possono scoraggiare il feedback. Uno storyboard disegnato a mano permette inoltre ai progettisti di sperimentare rapidamente diverse soluzioni e scenari, senza la pressione di dover creare disegni perfetti o complessi. Gli utenti e i membri del team possono concentrarsi esclusivamente sul contenuto del task, senza essere distratti da font, colori o icone.

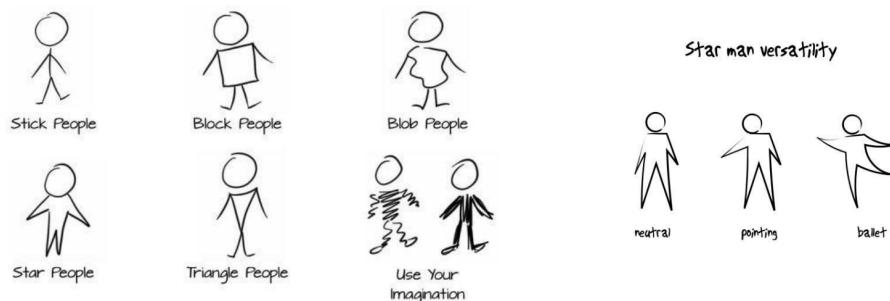


Figura 3.3: Sketching people

Per una singola applicazione, di solito si crea uno storyboard per 1-2 task principali. È importante non cercare di rappresentare ogni possibile task, poiché un errore comune è voler fare troppo e includere ogni dettaglio. Una buona pratica è ridurre il numero di storyboard a un massimo di 3-5 task, il che di solito corrisponde a 2-4 storyboard per progetto. Questo approccio permette di mantenere

il focus sui task più critici, senza perdere tempo su task secondari che potrebbero essere affrontati in seguito.

L'uso degli storyboard porta numerosi benefici nel processo di progettazione. Innanzitutto, enfatizzano come un'interfaccia aiuta a completare un task, mantenendo il focus della discussione sui compiti degli utenti, piuttosto che sui dettagli superficiali dell'interfaccia, come la posizione dei pulsanti. Gli storyboard sono un ottimo strumento per garantire che tutti i membri del team di progettazione abbiano una visione condivisa sugli obiettivi dell'app, evitando discussioni inutili su dettagli tecnici.

Capitolo 4

Prototyping

Il **prototyping** è una fase fondamentale del processo di design nell'Interazione Uomo-Macchina (HCI). Rappresenta un'opportunità per esplorare e testare diverse soluzioni, valutare l'usabilità dell'interfaccia e raccogliere feedback dagli utenti prima di implementare il sistema finale. Il prototyping non riguarda solo la creazione di modelli fisici o digitali, ma serve anche come base per validare idee e ipotesi attraverso un processo iterativo.

Dopo aver creato uno storyboard per visualizzare i task principali che un utente deve eseguire, il passo successivo è immaginare l'interazione e progettare l'interfaccia utente (UI). Questo significa pensare non solo all'aspetto visivo, ma anche agli strumenti disponibili, come i dispositivi hardware, i widget e gli stili di interazione. È importante che la UI sia progettata considerando l'ecosistema in cui sarà utilizzata, tenendo conto di dove, quando e con chi l'interazione avverrà.

Tuttavia, è normale che il progetto iniziale non sia perfetto. Gli errori fanno parte del processo di design, e per questo è cruciale testare le idee con utenti reali. Questo permette di verificare se la UI è completa, comprensibile e facile da usare.

4.1 Prototipo

Un prototipo è un modello di un sistema interattivo che simula o anima alcuni aspetti o funzionalità del sistema finale, ma non tutti. Il prototipo permette ai progettisti di sperimentare la progettazione senza sviluppare l'intero sistema. Può essere utilizzato per testare concetti e idee in un ambiente controllato e per raccogliere feedback prima della fase di sviluppo completa. Un esempio di prototipo potrebbe essere un mockup di uno smartphone, in cui vengono simulate solo alcune funzionalità, come la navigazione tra le schermate o l'interazione con i widget.

Esistono diversi modi di utilizzare i prototipi. In generale, essi vengono impiegati per:

- Provare e validare un'idea o un concetto: ad esempio, testare un nuovo sistema di navigazione per un'app.
- Imparare dagli utenti: raccogliere feedback diretto per migliorare il design.
- Sperimentare versioni intermedie o finali: i prototipi permettono di vedere come si evolve l'interazione con il sistema man mano che si aggiungono nuove funzionalità.

Esistono tre approcci principali al prototyping:

1. *Prototyping "throw away"*: Il prototipo viene creato per esplorare un'idea, e una volta ottenute le informazioni necessarie, viene scartato.
2. *Prototyping incrementale*: Le varie funzioni vengono aggiunte una alla volta al prototipo, permettendo di testare l'interfaccia in modo progressivo.
3. *Prototyping evolutivo*: Ogni nuovo prototipo si basa su quello precedente, migliorando gradualmente le funzionalità e il design.

4.2 Ciclo di prototipazione

Il design delle interfacce è raramente perfetto al primo tentativo. È fondamentale seguire un ciclo iterativo che prevede la creazione di un prototipo, la sua validazione con utenti reali e, in base ai risultati, la riprogettazione dell'interfaccia. Questo processo continua fino a ottenere un sistema che soddisfi le esigenze degli utenti. Il ciclo di prototyping segue una sequenza di progettazione, test e revisione.

Durante il processo di prototipazione iterativa, è utile utilizzare il concetto di hill climbing, che si riferisce al miglioramento continuo del progetto. Ogni iterazione del prototipo cerca di superare i difetti individuati nella versione precedente, migliorando gradualmente il design. Questo processo funziona solo se si riesce a capire a fondo cosa è sbagliato nel progetto e si propongono soluzioni mirate.

Uno dei problemi principali nel prototyping è il tempo necessario per realizzare i prototipi. Creare un prototipo richiede tempo, e se questo viene scartato, può sembrare tempo sprecato. Tuttavia, il prototyping rapido può aiutare a ridurre questa sensazione, purché non si sacrifichi la qualità del processo di valutazione. Un'altra sfida è la pianificazione. Poiché il processo di prototyping può essere imprevedibile, è difficile stabilire tempi e costi precisi. Spesso, il committente e il progettista devono negoziare continuamente per mantenere il progetto in linea con il budget. Infine, i prototipi possono trascurare alcune caratteristiche non funzionali, come la sicurezza, l'affidabilità o il tempo di risposta. Questi aspetti, se ignorati durante il prototyping, possono creare problemi nella fase di sviluppo finale.

Un ultimo problema da tenere presente è l'inerzia del design. Una volta presa una decisione progettuale, c'è il rischio che questa non venga mai più rivista, anche se si rivela errata. Il processo iterativo di prototipazione deve quindi includere una revisione costante delle scelte iniziali, per evitare di costruire l'interfaccia su fondamenta sbagliate. È essenziale individuare e affrontare le cause profonde dei problemi, piuttosto che limitarsi a correggere i sintomi.

Uno degli errori comuni durante il prototyping è focalizzarsi troppo presto sull'interfaccia utente (UI) senza aver prima compreso a fondo il task che l'utente deve completare. È importante quindi iniziare comprendendo bene i compiti e i bisogni dell'utente, per poi costruire la UI attorno a questi.

Esistono diverse tecniche di prototyping. Ogni tecnica ha vantaggi specifici a seconda del contesto in cui viene utilizzata e degli obiettivi del team di design.

4.3 Low fidelity prototypes

I low fidelity prototypes sono utilizzati nelle prime fasi del processo di progettazione. Si tratta spesso di modelli realizzati su carta, come i mock-up disegnati a mano, che possono includere più fogli di diverse dimensioni per rappresentare vari stati dell'interfaccia. Questi prototipi sono strumenti rapidi per esplorare le prime idee e testare l'interazione utente, consentendo di individuare e risolvere problemi di design senza investire troppo tempo o risorse.

4.3.1 Paper prototypes

Un tipo di low fidelity prototypes è il paper prototype, dove un'interfaccia disegnata a mano simula l'aspetto visivo di un'app. Le diverse schermate e gli elementi interattivi (come finestre, menu o dialog box) possono essere rappresentati da pezzi di carta che vengono posizionati e manipolati manualmente, simulando un'interazione reale. Ad esempio, un dito che tocca uno schermo simulato equivale a un clic del mouse, e la scrittura su un pezzo di carta rappresenta l'inserimento di testo. Una persona funge da "computer", simulando le risposte del sistema, come aggiornamenti di finestre o feedback, interagendo con i vari pezzi di carta. Questo tipo di prototipo, sebbene a bassa fedeltà in termini di aspetto visivo, può raggiungere un'elevata fedeltà nella simulazione delle interazioni e del flusso di lavoro.

Il paper prototyping richiede pochi materiali di base: carta, fogli trasparenti, penne, pennarelli, post-it, forbici e nastro adesivo. Questi strumenti semplici permettono di costruire prototipi in modo rapido ed economico, favorendo la sperimentazione continua. Utilizzando stencil per elementi dell'interfaccia utente (UI) o stampando schermate già esistenti, è possibile creare componenti riutilizzabili che velocizzano ulteriormente il processo.

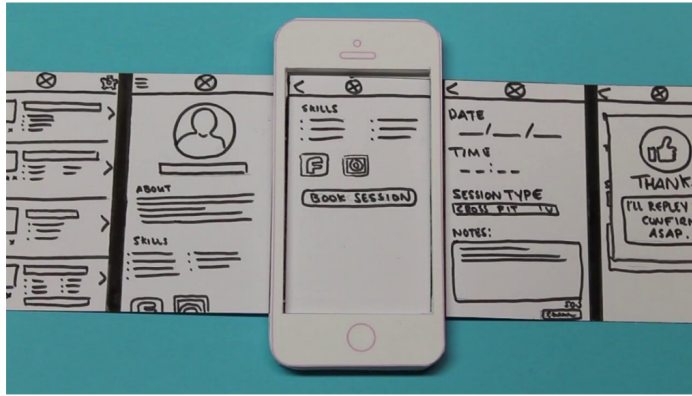


Figura 4.1: Esempio di paper prototype

Il principale vantaggio del paper prototyping è la sua rapidità. Schizzare su carta è molto più veloce che programmare, e modificare il design tra un test e l'altro è altrettanto semplice. Il prototyping su carta permette di concentrarsi sugli aspetti più importanti del design, senza perdersi nei dettagli tecnici. Gli utenti e i clienti possono partecipare attivamente alla discussione, fornendo feedback creativi e non limitati a questioni estetiche o funzionali minori. Questo approccio rende il paper prototyping accessibile anche ai non programmatori, dato che richiede solo abilità di base.

Durante il test di un paper prototype, il team di progettazione assume vari ruoli. Un membro del team funge da computer, simulando il comportamento del sistema, ma senza fornire feedback. Il facilitatore presenta l'interfaccia e i task all'utente, incoraggiandolo a pensare ad alta voce e descrivere le proprie azioni e pensieri mentre interagisce con il prototipo. L'osservatore si limita a prendere appunti senza intervenire, annotando dettagli importanti come le reazioni dell'utente, le difficoltà incontrate o i suggerimenti spontanei.

Il testing dei paper prototypes consente di raccogliere informazioni importanti. Ad esempio, permette di valutare se gli utenti comprendono il modello concettuale dell'interfaccia e se la funzionalità proposta soddisfa i loro bisogni. Il test rivela anche quanto sia intuitiva la navigazione, se gli utenti riescono a orientarsi facilmente tra le varie schermate e se la terminologia utilizzata è chiara. Inoltre, è possibile valutare i contenuti visivi delle schermate, identificando quali informazioni devono essere presenti per garantire un'interazione fluida.

Tuttavia, ci sono anche dei limiti. Con un paper prototype non è possibile testare aspetti come i colori, i font, il layout o l'efficienza dell'interfaccia. Inoltre, gli utenti tendono a essere più deliberati e meno esplorativi quando interagiscono con un prototipo di carta, sapendo che stanno lavorando con un modello non finito.

4.4 Medium fidelity prototypes

I prototipi a media fedeltà rappresentano una fase avanzata del processo di progettazione rispetto ai prototipi su carta, poiché offrono una visione più dettagliata dell'interfaccia e del layout. Tuttavia, non raggiungono la completezza dei prototipi ad alta fedeltà in termini di comportamento e interattività. Sono particolarmente utili per simulare l'interazione dell'utente con il sistema, testando la navigazione tra le schermate e il flusso generale senza concentrarsi su aspetti grafici o estetici.

Questi prototipi vengono realizzati tramite software interattivi che simulano l'aspetto di un'interfaccia utente, accettano alcuni input e rispondono agli utenti passando da una schermata all'altra. Sono spesso definiti mockup o wireframe interattivi, poiché rappresentano una versione semplificata dell'interfaccia.

La grafica nei prototipi a media fedeltà è stilizzata e spesso in bianco e nero o in scala di grigi, con l'intento di trasmettere l'impressione che il design sia ancora in fase preliminare. Ciò permette agli utenti e ai designer di concentrarsi sugli aspetti funzionali e strutturali del sistema, senza distrarsi con dettagli estetici come i colori o i font.

Un aspetto importante nello sviluppo di prototipi a media fedeltà è l'uso di librerie di design per interfacce utente (UI). Queste librerie forniscono una serie di componenti predefiniti e riutilizzabili, come pulsanti, menu e barre di navigazione, che semplificano la creazione di mockup interattivi. Le

librerie aiutano a mantenere una coerenza visiva e di interazione tra le varie schermate, rendendo più facile e veloce la progettazione del prototipo.



Ad esempio, potrebbero concentrarsi troppo sui colori, sui font o sulla grafica piuttosto che sul fatto che l'interfaccia risponda efficacemente ai loro bisogni.

Uno degli aspetti che si possono valutare con i prototipi ad alta fedeltà è il layout delle schermate. Questo permette di capire se gli utenti trovano le informazioni importanti con facilità o se l'interfaccia risulta eccessivamente complessa o disordinata.

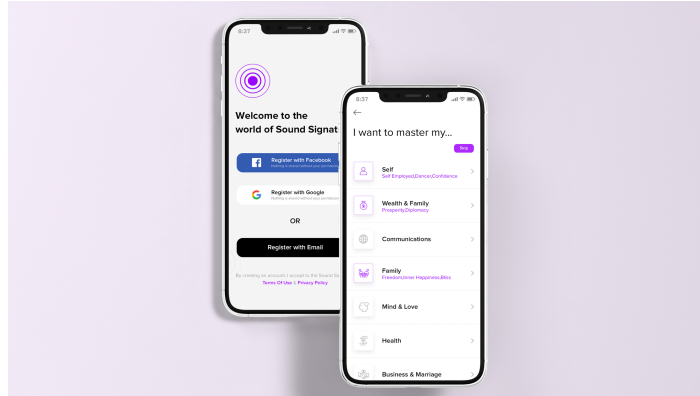


Figura 4.3: High fidelity prototype

Un altro elemento che può essere valutato riguarda colori, font e icone. Questi elementi visivi sono cruciali per la user experience (UX) e per la percezione dell'interfaccia da parte dell'utente. Anche il feedback interattivo può essere testato: ci si può chiedere se gli utenti notano e rispondono correttamente ai messaggi della barra di stato, ai cambiamenti di cursore o ad altri segnali visivi che indicano l'avanzamento delle operazioni.

I problemi di efficienza sono un'altra area di valutazione. Ad esempio, si può osservare se i controlli sono abbastanza grandi da essere cliccati comodamente, se gli elementi interattivi sono disposti in modo logico o se liste di scorrimento troppo lunghe creano frustrazione. In breve, un prototipo ad alta fedeltà consente di esaminare dettagli molto specifici della UI, che possono avere un impatto importante sull'esperienza utente complessiva.

Esistono diversi strumenti per la creazione di prototipi ad alta fedeltà. Tra i più popolari vi sono InVision e Figma, che permettono di creare interfacce altamente interattive e realistiche. Questi strumenti permettono di creare prototipi senza scrivere codice, rendendo possibile la creazione di layout e interfacce completamente funzionanti.

4.5.1 Metodo "Wizard of Oz"

Una tecnica particolare per simulare tecnologie non ancora completamente implementate è il Wizard of Oz. Questo approccio consente di testare un'applicazione che sembra completa, con interfaccia e algoritmi finalizzati, ma senza dover scrivere tutto il codice necessario. Un essere umano simula il comportamento del sistema, controllando l'interfaccia dietro le quinte, come un mago nascosto dietro un sipario. Questo metodo viene spesso utilizzato per testare applicazioni che utilizzano tecnologie complesse o non ancora pronte, come il riconoscimento vocale o l'intelligenza artificiale.

Per implementare questa tecnica, è necessario scegliere task specifici da simulare, creando mock-up dell'interfaccia che consentano al "mago" di intervenire nei punti critici. Un esempio pratico potrebbe riguardare lo sviluppo di un'app di assistente virtuale, dove il sistema di riconoscimento vocale non è ancora implementato. Il mago, nascosto dietro il sistema, risponde manualmente alle richieste dell'utente, simulando l'algoritmo di risposta automatica.

Questo approccio ha diversi vantaggi. Innanzitutto, è più veloce e meno costoso rispetto allo sviluppo di un prototipo completamente interattivo. Inoltre, consente di creare diverse varianti del prototipo in tempi rapidi, testando scenari che sarebbero difficili da implementare con un codice reale. Il "mago" può anche aiutare a identificare bug o problemi di design, fornendo al team di progettazione una comprensione più dettagliata di come dovrebbero funzionare gli algoritmi finali.

Nonostante i vantaggi, il metodo Wizard of Oz presenta alcuni rischi. La simulazione manuale può portare a ottimismo eccessivo riguardo alle capacità future del sistema. Ad esempio, un sistema di riconoscimento vocale simulato dal mago potrebbe funzionare senza errori, mentre un sistema reale potrebbe avere tassi di errore significativi. Inoltre, il comportamento del mago è difficile da

replicare esattamente come farebbe una macchina, e potrebbe rispondere in modo più efficiente di quanto un sistema reale riuscirebbe a fare.

Capitolo 5

Tecniche di valutazione

5.1 Metodi di osservazione

I metodi osservazionali sono tecniche impiegate per valutare l'usabilità e la funzionalità di un sistema interattivo, testando direttamente il modo in cui gli utenti reali utilizzano l'interfaccia. A differenza delle valutazioni teoriche, l'osservazione richiede un artefatto concreto: può trattarsi di una simulazione, di un prototipo o del sistema completo. Questi test possono svolgersi in laboratori, sul campo o in collaborazione con gli utenti e offrono informazioni preziose sia sull'aspetto progettuale che sull'implementazione. Idealmente, la valutazione dovrebbe avvenire in tutte le fasi del ciclo di vita del design per garantire una migliore aderenza alle esigenze dell'utente finale.

La valutazione ha tre obiettivi principali:

1. verificare l'efficacia delle funzionalità del sistema,
2. valutare l'impatto dell'interfaccia sull'utente,
3. identificare eventuali problemi specifici di usabilità.

Questi obiettivi aiutano a comprendere se il sistema risponde ai bisogni dell'utente e se ci sono aree da migliorare.

Quando si conduce una valutazione, è importante selezionare utenti rappresentativi del target, con conoscenze e competenze simili, per ottenere dati utili e rilevanti. La numerosità del campione è cruciale: secondo l'esperto di usabilità Jakob Nielsen, è consigliabile includere almeno 3-5 utenti per ottenere feedback significativo.

L'approccio con scenario prevede la simulazione di una situazione reale in cui l'utente si immedesima. Lo scenario permette di definire i contesti, le azioni e gli strumenti coinvolti. I task, invece, sono i compiti specifici che l'utente è chiamato a svolgere, come ad esempio eseguire una ricerca o completare una transazione online. Alcune valutazioni prevedono anche task liberi, in cui si lascia l'utente esplorare liberamente il sistema.

La valutazione può avvenire in laboratorio o sul campo. In laboratorio, i progettisti hanno accesso a strumentazioni specializzate e possono condurre i test in un ambiente privo di distrazioni, ma questo contesto manca di autenticità e rende difficile osservare interazioni collaborative tra utenti. Al contrario, i test sul campo mantengono intatto il contesto naturale e consentono di effettuare studi longitudinali, ma presentano sfide come rumori di fondo e distrazioni.

I metodi osservazionali sono per loro natura soggettivi, poiché dipendono dalla competenza del valutatore nel riconoscere i problemi e interpretare il comportamento dell'utente. Esiste inoltre il rischio di bias del valutatore, che può influenzare inconsapevolmente le osservazioni. Questo rischio può essere ridotto coinvolgendo più valutatori per ottenere una visione più equilibrata e affidabile.

Le misure prodotte da questi metodi osservazionali sono generalmente qualitative e non numeriche, ma offrono dettagli ricchi che non emergerebbero da misure quantitative. I dati qualitativi consentono di esplorare le esperienze degli utenti, le emozioni e i problemi riscontrati, fornendo una base più approfondita per miglioramenti progettuali.

Durante la valutazione osservazionale, è essenziale rispettare l'anonimato dei partecipanti, utilizzando identificativi anonimi e informazioni aggregate. In alcuni casi, è opportuno prevedere una ricompensa per i partecipanti, per compensare il tempo dedicato alla valutazione e incentivare una partecipazione più attiva e sincera.

5.1.1 Think aloud

Nel metodo Think Aloud (pensare ad alta voce), l'utente è osservato mentre svolge i task e viene invitato a descrivere verbalmente cosa sta facendo e perché. Questa tecnica, semplice e senza bisogno di competenze avanzate, fornisce informazioni dirette sull'esperienza d'uso e su come il sistema viene realmente utilizzato. Tuttavia, è soggettiva e può influenzare le prestazioni dell'utente, poiché il parlare richiede uno sforzo cognitivo aggiuntivo.

5.1.2 Cooperative evaluation

La Cooperative Evaluation è una variante del Think Aloud, in cui l'utente e il valutatore collaborano e possono fare domande a vicenda. Questo metodo, meno rigido e più informale, incoraggia l'utente a criticare il sistema e consente chiarimenti in tempo reale, offrendo un feedback immediato e dettagliato.

5.1.3 Protocol analysis

La Protocol Analysis implica la registrazione delle sessioni di valutazione per consentire un'analisi approfondita successiva. Le modalità di registrazione possono includere l'uso di carta e penna, audio, video o log su computer. Ciascun metodo ha vantaggi e svantaggi: ad esempio, l'audio è ottimo per il Think Aloud, ma difficile da sincronizzare con altre fonti, mentre i video offrono realismo ma sono più invasivi e richiedono attrezzature specifiche. I log su computer raccolgono molti dati automaticamente, ma la quantità può rendere l'analisi complessa. Le trascrizioni richiedono competenze specifiche e talvolta supporto tecnologico.

5.1.4 Post-task walkthroughs

Nella tecnica dei Post-task Walkthroughs, la registrazione della sessione viene riprodotta per il partecipante, che può commentare e spiegare le proprie azioni. Se immediato, il feedback è fresco nella memoria dell'utente; se ritardato, il valutatore ha tempo di formulare domande mirate. Questo metodo è utile per approfondire i motivi dietro le scelte e le alternative considerate dall'utente, risultando particolarmente valido quando il Think Aloud non è fattibile.

5.2 Esperimenti

La valutazione sperimentale è un metodo fondamentale per esaminare aspetti specifici del comportamento interattivo degli utenti con un sistema. A differenza dei metodi osservazionali, la valutazione sperimentale permette di condurre test in condizioni controllate, con l'obiettivo di valutare l'effetto di una o più variabili sulle prestazioni degli utenti. In un esperimento, il valutatore stabilisce delle ipotesi da verificare e sceglie le variabili da modificare o misurare, suddividendo i partecipanti in gruppi che sperimentano condizioni diverse.

Per progettare un esperimento efficace, ci sono alcuni fattori chiave da considerare:

- *Partecipanti*: È essenziale selezionare utenti rappresentativi del target del sistema, e avere un campione di dimensioni sufficienti per ottenere dati significativi.
- *Variabili*: Si distinguono le variabili indipendenti (IV), che rappresentano i fattori da modificare tra le diverse condizioni, come lo stile dell'interfaccia o il numero di elementi in un menu, e le variabili dipendenti (DV), che sono le misure raccolte durante l'esperimento, come il tempo di completamento di un task o il numero di errori.
- *Ipotesi*: L'ipotesi è un'affermazione provvisoria che il test deve validare o confutare.

Nella progettazione sperimentale, due tipi di ipotesi sono centrali:

1. *Ipotesi alternativa*: Propone che la variazione di una variabile indipendente causi un cambiamento misurabile nella variabile dipendente. Ad esempio, *"il tasso di errore aumenta al diminuire della dimensione del font"*.

2. *Ipotesi nulla*: Afferma che non ci sarà alcun cambiamento nella variabile dipendente a seguito della variazione della variabile indipendente, ad esempio, *"non ci sarà alcuna differenza nel tasso di errore cambiando la dimensione del font"*.

Lo scopo dell'esperimento è spesso quello di provare a rigettare l'ipotesi nulla per confermare l'effetto della variabile indipendente.

Per creare un design sperimentale efficace, il valutatore deve prendere diverse decisioni. Dopo aver definito ipotesi, variabili e partecipanti, il prossimo passo è selezionare il metodo sperimentale. Esistono due approcci principali:

1. *Between Subjects*: In questo metodo, ogni partecipante sperimenta solo una condizione. Richiede un numero maggiore di partecipanti e può risultare meno influenzato dal "trasferimento dell'apprendimento", ma i risultati potrebbero variare a causa delle differenze tra i partecipanti stessi.
2. *Within Subjects*: In questo metodo, ogni partecipante sperimenta tutte le condizioni, eseguendo quindi il test più volte. È meno costoso poiché richiede meno partecipanti e le differenze individuali incidono meno sui risultati. Tuttavia, risente del "trasferimento dell'apprendimento", poiché l'utente potrebbe migliorare nel test solo grazie alla ripetizione.

Per ridurre l'effetto del trasferimento dell'apprendimento nei test within subjects, si possono alterare le condizioni tra i partecipanti. Ad esempio, una metà inizia con la condizione di controllo, passando poi a quella sperimentale, mentre l'altra metà segue il percorso opposto. Questo approccio aiuta a ridurre i bias derivanti dall'ordine di esposizione alle condizioni.

Una volta completato l'esperimento, i dati raccolti dalla variabile dipendente vengono analizzati statisticamente. È consigliabile esaminare graficamente i dati per identificare possibili outliers e salvare una copia dei dati originali per futuri approfondimenti.

Durante la fase di analisi statistica, è utile distinguere tra variabili discrete e continue. Le variabili continue, come il tempo di completamento di un task, assumono valori superiori a zero e possono essere rappresentate su una scala numerica. Le variabili indipendenti sono generalmente discrete, poiché rappresentano categorie o condizioni separate.

La scelta dei metodi statistici dipende dalla distribuzione dei dati:

- Se i dati presentano una distribuzione normale, si possono usare test parametrici per analizzare le differenze tra i gruppi.
- Se i dati non seguono una distribuzione normale, si preferiscono test non parametrici, come la tabella di contingenza.

L'analisi si conclude con la verifica delle ipotesi. Si valuta se i risultati permettono di accettare o rigettare l'ipotesi nulla. Per esempio, si potrebbe concludere che "al 99% possiamo rigettare l'ipotesi nulla", indicando un'elevata probabilità che l'ipotesi alternativa sia corretta.

5.2.1 A/B testing

L'A/B testing è una delle tecniche di valutazione più semplici ed efficaci nell'ambito dell'usabilità. Consiste in un esperimento in cui viene testata una sola variabile indipendente che ha due valori distinti, indicati come A e B. Questi due valori rappresentano due diverse condizioni di un'interfaccia o di un elemento dell'interfaccia. Gli utenti vengono casualmente esposti a una delle due condizioni e i risultati delle interazioni vengono misurati e confrontati.

Questo metodo viene largamente utilizzato per i test online, specialmente su siti web, dove ogni utente che accede può visualizzare una delle due versioni (A o B) a caso. Questo permette di raccogliere dati quantitativi su come gli utenti interagiscono con le diverse versioni e di identificare quale delle due condizioni è più efficace in termini di usabilità.

Come per ogni esperimento, l'A/B testing parte dalla definizione di un'ipotesi. Esistono due ipotesi principali:

1. *Ipotesi nulla*: non c'è differenza significativa tra le condizioni A e B. In questo caso, i risultati ottenuti non dimostrano un vantaggio specifico per una delle due varianti.

2. *Ipotesi alternativa*: esiste una differenza significativa tra le condizioni A e B, e quindi una delle due condizioni risulta superiore. L'obiettivo dell'A/B testing è spesso quello di confermare l'ipotesi alternativa e dimostrare che una delle due condizioni porta a risultati migliori.

Per verificare queste ipotesi si utilizza solitamente il t-test, un test statistico che si basa sulla distribuzione Gaussiana (o normale). Questo test permette di misurare se le differenze tra i risultati delle due condizioni sono statisticamente significative, e quindi di determinare con maggiore sicurezza quale delle due varianti è preferibile.

L'A/B testing presenta diversi vantaggi. Uno dei principali è la semplicità: progettare e implementare un A/B test richiede pochi passaggi e può essere eseguito rapidamente. Inoltre, i risultati sono generalmente facili da misurare e interpretare, offrendo un feedback chiaro su quale variante ha ottenuto i migliori risultati. Questo lo rende particolarmente utile per prendere decisioni basate su dati concreti piuttosto che su ipotesi o preferenze soggettive.

Tuttavia, l'A/B testing ha anche alcuni limiti. È adatto solo per situazioni in cui sono presenti due condizioni distinte. Quando è necessario testare più di due varianti, si ricorre a tecniche come il multivariate testing. Inoltre, non è sempre facile definire quale aspetto specifico misurare tra le due condizioni, poiché ogni cambiamento può influenzare vari fattori del comportamento utente.

5.3 Basato su esperti

Nel design dell'interazione uomo-macchina, le tecniche di valutazione basate su esperti offrono un modo per identificare potenziali problemi senza coinvolgere direttamente gli utenti. Questo approccio permette a uno o più esperti di valutare un'interfaccia, considerando come potrebbe essere percepita e utilizzata dall'utente tipico. Queste tecniche si basano su principi teorici e empirici dell'usabilità e della psicologia cognitiva, consentendo di evidenziare problemi che violano regole note o principi fondamentali dell'interazione uomo-macchina.

Le valutazioni basate su esperti presentano diversi vantaggi. Innanzitutto, sono meno costose rispetto a quelle che coinvolgono utenti reali e possono essere utilizzate in ogni fase del progetto, anche nelle fasi iniziali, quando l'interfaccia è ancora in fase di sviluppo. Tuttavia, non sono in grado di rappresentare il vero uso del sistema, ma solo di misurare l'aderenza a principi e linee guida conosciute. Di conseguenza, potrebbero non evidenziare problemi specifici dell'interazione reale.

Esistono diverse metodologie di valutazione basate su esperti, ognuna con specifiche caratteristiche e aree di applicazione.

5.3.1 Cognitive walkthrough

Il Cognitive Walkthrough è una tecnica proposta da Polson et al. nel 1992, mirata a valutare quanto il design di un'interfaccia supporti l'utente nell'apprendimento di un task. Questo metodo è spesso condotto da esperti in psicologia cognitiva, i quali esaminano il design passo dopo passo per identificare eventuali difficoltà o incoerenze cognitive. Per effettuare un cognitive walkthrough, sono necessari dettagli sul sistema, la descrizione del task da svolgere, le caratteristiche dell'utente tipo e una sequenza dei passaggi da eseguire.

Durante il walkthrough, per ogni passo, l'esperto risponde a domande chiave come:

- L'effetto dell'azione è lo stesso dell'obiettivo dell'utente?
- L'utente può vedere che l'azione è disponibile?
- L'utente capirà che quella è l'azione da eseguire?
- Dopo averla eseguita, capirà il feedback ricevuto?

5.3.2 Heuristic evaluation

L'Heuristic Evaluation, sviluppata da Jakob Nielsen e Rolf Molich, è una tecnica in cui gli esperti valutano un'interfaccia in base a una serie di euristiche – principi generali e regole pratiche per il design dell'usabilità. Ogni violazione delle euristiche viene registrata, classificata in base alla severità, e contribuisce a identificare problemi di usabilità.

Le dieci euristiche di Nielsen includono:

1. Visibilità dello stato del sistema
2. Corrispondenza tra il sistema e il mondo reale
3. Libertà e controllo da parte degli utenti
4. Coerenza e standard
5. Prevenzione degli errori
6. Riconoscere piuttosto che ricordare
7. Flessibilità ed efficienza d'uso
8. Design minimalista ed estetico
9. Aiutare gli utenti a riconoscere, diagnosticare e correggere gli errori
10. Aiuto/guida e documentazione

5.3.3 Model-Based evaluation

La model-based evaluation si basa su modelli teorici, come GOMS (Goals, Operators, Methods, Selections), KLM (Keystroke-Level Model) e Dialog Models, per prevedere le prestazioni e valutare l'interazione tra l'utente e il sistema. Questi modelli permettono di stimare il tempo di completamento di task specifici o di identificare sequenze di dialogo problematiche, offrendo un approccio strutturato per prevedere e prevenire difficoltà.

5.3.4 Review-Based evaluation

La Review-Based Evaluation utilizza i risultati di studi precedenti per valutare e informare le decisioni progettuali. Si tratta di una tecnica che richiede la consulenza di un esperto per assicurare che i risultati siano rilevanti e trasferibili al nuovo contesto di design. Questa tecnica è particolarmente utile per verificare assunzioni o miglioramenti rispetto a sistemi simili.

Capitolo 6

Sviluppo di sistemi interattivi

Nel mondo del design di sistemi interattivi, la metodologia utilizzata per gestire il progetto è cruciale per determinare il successo del prodotto finale. Esistono vari modelli di gestione dei progetti, ma due delle metodologie più ampiamente utilizzate sono il modello Waterfall e il metodo Agile. Ciascuno di questi approcci ha caratteristiche uniche, vantaggi e svantaggi, e viene scelto in base alla natura del progetto e alle esigenze del team.

6.1 Modello waterfall

Il modello Waterfall (a cascata) è uno dei modelli più tradizionali e segue una sequenza lineare e ben definita. Ogni fase deve essere completata prima che la successiva possa iniziare, creando una struttura rigida e chiara per lo sviluppo del progetto.

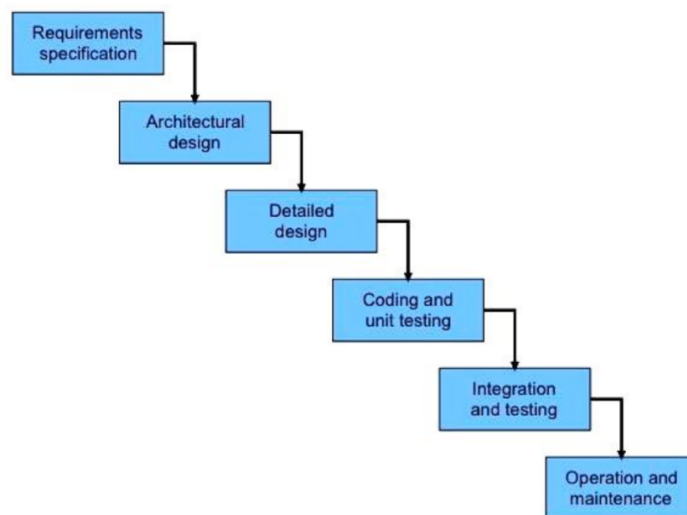


Figura 6.1: Modello waterfall

Le fasi principali del modello Waterfall includono:

1. *Analisi dei requisiti*: in questa fase vengono raccolti i requisiti dettagliati del sistema, definendo cosa deve fare il sistema e quali sono le aspettative degli utenti.
2. *Progettazione*: una volta definiti i requisiti, si passa alla progettazione del sistema, dove viene creato un modello dettagliato che includa aspetti come l'architettura del sistema e il design dell'interfaccia utente.
3. *Implementazione*: il sistema viene costruito seguendo il progetto definito, con gli sviluppatori che traducono il design in codice.
4. *Testing*: una volta completato lo sviluppo, il sistema viene testato per verificare che funzioni correttamente e soddisfi i requisiti stabiliti.

5. *Manutenzione*: dopo il rilascio, il sistema entra in fase di manutenzione, in cui vengono apportati miglioramenti e corretti eventuali bug.

Un vantaggio del modello Waterfall è la sua struttura chiara: le fasi sequenziali e ben definite lo rendono ideale per progetti in cui i requisiti sono stabili e non prevedono cambiamenti significativi durante lo sviluppo. Tuttavia, la rigidità del Waterfall può essere uno svantaggio in contesti in cui è richiesta flessibilità, poiché ogni modifica nei requisiti implica la ripetizione di diverse fasi.

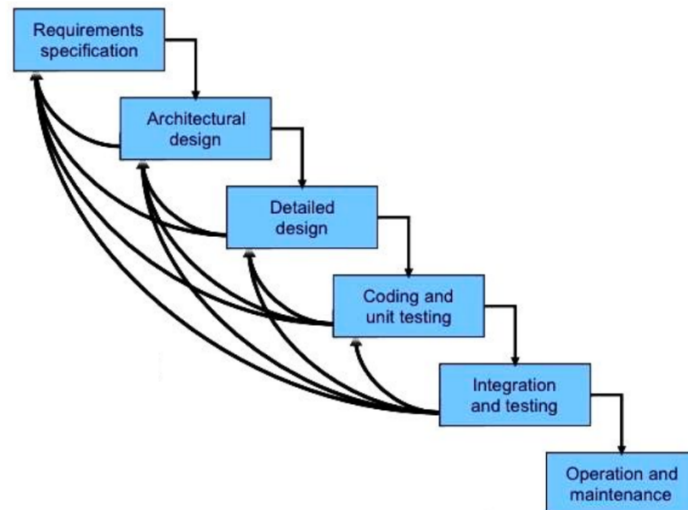


Figura 6.2: Ciclo di vita per sistemi interattivi

6.2 Metodo agile

Il metodo Agile è stato introdotto per rispondere alla necessità di maggiore flessibilità nel processo di sviluppo. A differenza del modello Waterfall, Agile suddivide il progetto in cicli iterativi e incrementali chiamati sprint. Ogni sprint rappresenta una breve fase di sviluppo in cui vengono progettate, sviluppate e testate funzionalità specifiche. Alla fine di ogni sprint, il team verifica i progressi fatti, raccoglie feedback e adatta il progetto alle esigenze emergenti.

Le caratteristiche principali del metodo Agile includono:

- *Iterazione continua*: ogni ciclo fornisce un prodotto funzionante con nuove funzionalità o miglioramenti.
- *Collaborazione costante con gli utenti*: il feedback viene raccolto alla fine di ogni sprint, consentendo aggiustamenti immediati.
- *Flessibilità e adattabilità*: i team possono rispondere rapidamente ai cambiamenti nei requisiti e agli input degli stakeholder.

Agile si basa su quattro principi fondamentali:

1. Focus sugli individui e le interazioni piuttosto che sui processi e strumenti.
2. Software funzionante più che documentazione dettagliata.
3. Collaborazione con il cliente più che negoziazione del contratto.
4. Risposta al cambiamento più che seguire un piano fisso.

Questi principi mettono l'accento sull'adattabilità e sulla capacità di rispondere rapidamente a nuove esigenze, rendendo Agile ideale per progetti complessi e dinamici in cui le specifiche possono cambiare nel tempo.

6.2.1 Agile UDC

La progettazione di sistemi interattivi richiede oggi un approccio che coniughi il metodo Agile con i principi del User-Centered Design (UCD). Mentre Agile offre flessibilità e iterazione continua, UCD assicura che il design sia incentrato sui bisogni degli utenti. Questo connubio tra Agile e UCD permette di creare sistemi che rispondano ai requisiti in modo dinamico, mantenendo sempre al centro l'esperienza utente.

Integrare Agile e UCD significa bilanciare l'attenzione sull'utente finale con la necessità di flessibilità del progetto. In un contesto Agile UCD, i team lavorano su brevi sprint che includono non solo fasi di sviluppo ma anche test e valutazioni con gli utenti. In ogni ciclo, si creano prototipi e versioni preliminari del sistema, che vengono testate dagli utenti e modificate secondo il feedback raccolto. Questo approccio riduce i rischi di progettazione, permettendo correzioni frequenti e incrementali basate su dati reali.

La combinazione di Agile e UCD richiede strumenti e processi che facilitino la collaborazione tra sviluppatori, designer e utenti. Ad esempio, per mantenere l'attenzione sull'utente, si utilizzano frequentemente storyboards e personas, che rappresentano il punto di vista dell'utente all'interno del processo di sviluppo.

Un altro strumento utile sono i wireframe, che permettono di esplorare l'aspetto e la navigazione del sistema senza sviluppare completamente il codice. I test di usabilità vengono eseguiti alla fine di ogni sprint, consentendo agli sviluppatori di adattare il sistema in base al feedback immediato.

L'integrazione di Agile e UCD presenta molti vantaggi, tra cui:

- *Adattabilità e flessibilità*: il sistema può evolvere secondo i feedback degli utenti, adattandosi ai cambiamenti nelle esigenze.
- *Rilevanza per l'utente*: grazie all'approccio centrato sull'utente, il sistema risponde meglio alle aspettative e ai bisogni dell'utente finale.
- *Riduzione dei rischi*: le iterazioni frequenti permettono di rilevare e correggere errori nelle fasi iniziali, riducendo i costi e i tempi di modifica.

Capitolo 7

Interfacce

7.1 Modi

Nel design dell'interazione uomo-macchina, i **modi** rappresentano uno dei concetti chiave per la gestione delle risposte del sistema ai comandi dell'utente. Un'interfaccia è definita "modale" quando il significato di un gesto cambia a seconda dello stato attuale del sistema.

I modi determinano come un'interfaccia interpreta un'azione specifica dell'utente, come un clic o un gesto, variando il suo comportamento a seconda delle impostazioni o dei contesti attivi.

Prima di analizzare i modi, è essenziale comprendere alcuni termini chiave:

- *Content*: rappresenta l'insieme delle informazioni presenti nel sistema, significative e utili per l'utente.
- *GID (Graphical Input Device)*: è un dispositivo grafico di input, come il mouse, che permette di comunicare la posizione del cursore o la selezione di un oggetto.
- *GID Button*: è il tasto principale del GID, utilizzato per selezionare o attivare elementi.
- *Tap*: si riferisce all'azione di premere e rilasciare un tasto, come il tocco su uno schermo o la pressione su un mouse.
- *Click, Double-Click, Drag e Swipe*: questi termini indicano azioni di interazione comune. Ad esempio, il click rappresenta una singola pressione, mentre il drag prevede il trascinamento di un oggetto mantenendo premuto il pulsante.

Una *gesture* (o gesto) rappresenta una sequenza di azioni completata automaticamente una volta avviata. I modi descrivono il modo in cui un'interfaccia risponde ai gesti. Se un'interfaccia interpreta un gesto sempre nello stesso modo, allora si trova in un unico modo. Tuttavia, se l'interpretazione del gesto cambia a seconda dello stato del sistema, ci troviamo di fronte a un'interfaccia in modalità variabile. Ad esempio, il tasto "Invio" potrebbe servire per andare a capo in un documento o per confermare un comando in una finestra di dialogo. L'interfaccia, quindi, assume significati diversi per lo stesso gesto, a seconda dello stato in cui si trova.

Esempio 7.1.1 – Torcia e sveglia

Prendiamo ad esempio una torcia che si accende e si spegne con un pulsante a pressione.

Quando è in tasca, non possiamo prevedere con sicurezza cosa accadrà premendo il pulsante, poiché non è visibile lo stato del sistema (accesa o spenta). Questo richiede una verifica separata dello stato della torcia.

Un altro esempio comune è la sveglia con un pulsante per attivarla o disattivarla.

Se la sveglia è attiva e si preme il pulsante, questa si disattiva, e viceversa. Al buio, non vedendo il display, non è possibile conoscere lo stato attuale della sveglia e prevedere l'effetto della pressione del pulsante.

Il *luogo dell'attenzione* è un concetto che rappresenta l'oggetto fisico o mentale su cui l'utente concentra intenzionalmente la sua attenzione. È importante notare che l'utente può avere un solo luogo

dell'attenzione alla volta. Se un cambiamento o un'azione si verifica al di fuori di questo luogo, è possibile che l'utente non se ne accorga, causando potenziali errori.

Esempio 7.1.2 – Caps lock

Il tasto Caps Lock crea un modo: quando attivato, tutte le lettere digitate sono maiuscole. È un errore comune iniziare a scrivere e accorgersi solo dopo alcune parole che il blocco maiuscole è attivo. Sebbene un LED verde sul tasto stesso segnali chiaramente lo stato attuale, e alcune applicazioni visualizzino un indicatore nella barra di stato, l'errore persiste. Ciò accade perché l'utente si focalizza sul contenuto digitato, non sullo stato di Caps Lock, situato fuori dal suo luogo dell'attenzione.

7.1.1 Errori di modo

I modi possono essere utili in alcune situazioni per aumentare la flessibilità, ma possono anche portare a errori. L'esperienza di un utente non elimina la possibilità di errori legati ai modi, poiché l'esperto ha sviluppato abitudini che potrebbero renderlo più vulnerabile a fraintendimenti in caso di cambiamenti nella modalità.

Gli errori di modo, ossia gli errori causati da una valutazione sbagliata dello stato del sistema, possono essere ridotti seguendo alcuni principi:

1. Evitare i modi quando possibile.
2. Segnalare chiaramente i modi presenti.
3. Distinguere i comandi in base ai modi: in modo che un comando emesso nel modo errato non produca effetti indesiderati.

Un'interfaccia è *modale* se

1. lo stato corrente dell'interfaccia non è il luogo dell'attenzione dell'utente e
2. se rispetto a un gesto può rispondere a quel gesto in diversi modi a seconda dello stato del sistema.

Invece, un'interfaccia è completamente *non-modale* se risponde sempre nello stesso modo a ogni gesto possibile. È possibile misurare il grado di "modularità" di un'interfaccia attraverso la probabilità che un gesto venga interpretato in modo non ambiguo, con 0 che indica un'interfaccia completamente modale e 1 un'interfaccia totalmente non modale.

Alcuni modi sono *modi temporanei* e svaniscono dopo l'uso (ad esempio, l'uso del pennello in Word attivato con un singolo clic), mentre altri sono *quasi-modi*, attivi finché si mantiene premuto un controllo, come il tasto Shift o CTRL.

7.1.2 Noun-verb e verb-noun

Le azioni in un'interfaccia possono essere strutturate in due modi:

1. *Noun-Verb*: si seleziona prima l'oggetto e poi si sceglie l'azione da eseguire su di esso.
2. *Verb-Noun*: si seleziona prima l'azione e poi l'oggetto.

Entrambi i metodi hanno vantaggi e svantaggi, influenzando il flusso naturale dell'attenzione dell'utente e la facilità di completamento dell'azione.

Ad esempio, cambiare il font di un testo rappresenta un'interazione noun-verb (prima si seleziona il testo, poi si applica il font). La selezione di un tool in una palette di strumenti è invece un esempio di verb-noun (prima si sceglie lo strumento, poi si applica all'oggetto).

Nell'approccio verb-noun, l'utente è spinto a focalizzare il luogo dell'attenzione sull'azione da compiere, selezionandola per prima. Una volta selezionata l'azione, l'utente sposta il suo focus sull'oggetto. Tuttavia, questo approccio può portare a errori di modo, specialmente se l'utente si distrae o il luogo dell'attenzione viene spostato in modo imprevisto. Questo tipo di interazione richiede, infatti, un doppio cambio del luogo dell'attenzione: dall'azione inizialmente selezionata all'oggetto su cui essa va applicata.

D'altra parte, l'approccio noun-verb si caratterizza per la selezione iniziale dell'oggetto che è già nel luogo dell'attenzione dell'utente. In questo caso, una volta scelto l'oggetto, il luogo dell'attenzione si sposta solo sull'azione, riducendo il numero di passaggi e spostamenti necessari per completare l'operazione. Questo tipo di sequenza offre vantaggi significativi in termini di semplicità e reversibilità dell'azione: interrompere un'operazione richiede soltanto di deselezionare l'oggetto, senza bisogno di confermare o annullare l'azione.

Esempio 7.1.3 – Sistema per richiedere prodotti

Consideriamo il caso di un sistema progettato per richiedere prodotti in una multinazionale.

Nel flusso originale, basato sul modello verb-noun, all'utente veniva richiesto di selezionare per prima cosa il dipartimento a cui i prodotti sarebbero stati inviati. Solo in seguito poteva scegliere gli articoli desiderati. Questo processo, seppur strutturato, portava a confusione e a errori. In particolare, il sistema impostava di default il dipartimento associato alla postazione in uso, ma se l'utente si trovava temporaneamente in un altro dipartimento, spesso dimenticava di cambiare l'impostazione, causando così l'invio degli articoli a un luogo sbagliato. Questo errore derivava dal fatto che l'ordine delle azioni non seguiva il flusso naturale dell'attenzione dell'utente, il quale si concentrava prima sugli articoli necessari e solo successivamente sul luogo di destinazione.

Per risolvere questi problemi, è stata introdotta una modifica al flusso, trasformandolo in un processo di tipo noun-verb. La soluzione prevedeva di mettere in cima all'interfaccia la scelta degli articoli, come primo passo dell'ordine. Inoltre, è stata aggiunta una descrizione chiara: "A quale dipartimento vuoi che gli articoli siano inviati?" per guidare meglio l'utente. Il sistema è stato inoltre modificato per identificare automaticamente il dipartimento dell'utente tramite il login, con l'opzione "Il mio dipartimento" come prima voce nella lista.

Questa nuova interfaccia ha portato diversi vantaggi. In primo luogo, la scelta del dipartimento è diventata un'azione finale, che completa logicamente il processo. Questo cambiamento ha contribuito a ridurre gli errori e a migliorare il flusso naturale dell'attenzione dell'utente, il quale non è più costretto a ricordare di cambiare il dipartimento predefinito. Infine, è stata eliminata la necessità di un click finale di conferma, riducendo così il numero di azioni richieste per completare l'ordine.

7.2 Affordances e signifiers

Il concetto di *affordance*, introdotto dallo psicologo Don Norman:

"una relazione tra le proprietà di un oggetto e le capacità dell'agente che determinano esattamente come l'oggetto potrebbe essere utilizzato"

In altre parole, la forma, la dimensione e l'aspetto dell'oggetto suggeriscono implicitamente quali azioni l'utente può compiere.



Figura 7.1: Affordance

L'affordance dipende sia dall'oggetto sia dall'utente. Se l'utente possiede le capacità percettive per comprendere come usare l'oggetto, allora l'affordance può guidarlo verso l'azione corretta. Tuttavia, in alcuni casi, le affordance non sono ovvie, o possono addirittura essere "nascoste", portando l'utente a fraintendimenti.

La famosa frase di Eleanor Gibson, "percepiamo per imparare, così come impariamo per percepire" illustra come l'interazione tra percezione e azione sia fondamentale. Questo principio viene evidenziato

dal celebre esperimento del “visual cliff” (baratro visivo), in cui i neonati venivano messi su una piattaforma trasparente sospesa sopra il suolo. L’esperimento dimostrava come la percezione del baratro induceva i bambini a evitare l’area, dimostrando la percezione immediata di un affordance che suggeriva pericolo.

Mentre le affordances sono caratteristiche intrinseche degli oggetti, i signifiers sono indicatori percepibili che comunicano all’utente quale comportamento adottare con un oggetto. Don Norman li definisce come

“qualsiasi segno, simbolo, suono o altro indicatore percepibile che guida l’utente verso l’azione corretta”

I signifiers includono elementi come etichette, frecce, simboli o icone che rendono esplicite le azioni desiderate. Ad esempio, un cartello con scritto “spingere” o “tirare” su una porta è un signifier che elimina ogni ambiguità su come aprirla.

Nelle interfacce uomo-macchina, i signifiers svolgono un ruolo fondamentale per guidare l’utente. Poiché l’interazione con i sistemi digitali può essere meno intuitiva rispetto agli oggetti fisici, i signifiers diventano più rilevanti delle affordances. I signifiers visivi e uditivi sono particolarmente utili per rendere chiaro cosa l’utente può fare in ogni momento, riducendo così l’incertezza.

Il rapporto tra affordances e signifiers può essere schematizzato con una tabella che mostra come la presenza o l’assenza di questi elementi influenzi l’interazione dell’utente.

Affordance	Signifier	Risultato
Presente	Presente	Affordance percepita e utilizzata correttamente.
Presente	Assente	Affordance nascosta; può creare confusione.
Assente	Presente	L’utente potrebbe essere indotto a fare azioni errate (falsa affordance).
Assente	Assente	Nessun indicatore di azione; l’utente non sa come procedere.

Tabella 7.1: Affordance vs signifiers

Questa classificazione aiuta a comprendere come progettare sistemi con affordances e signifiers in modo da garantire una buona usabilità. Ad esempio, quando un affordance non è immediatamente percepibile, l’aggiunta di un signifier adeguato può prevenire errori di interpretazione.

Capitolo 8

Design di app mobili

La **progettazione di applicazioni mobili** presenta una serie di sfide specifiche dovute alle caratteristiche dell'utente tipo e del contesto d'uso di questi dispositivi. Gli utenti di dispositivi mobili si trovano spesso in movimento, eseguono compiti nei brevi ritagli di tempo disponibili e, per questa ragione, non possono dedicare la loro attenzione in modo continuativo e concentrato come farebbero con un computer desktop.

L'interazione con un'app mobile è spesso interrotta da eventi esterni, come notifiche, chiamate o situazioni ambientali che richiedono attenzione. Inoltre, gli utenti possono trovarsi in situazioni in cui non devono disturbare altre persone, come in luoghi pubblici o al cinema.

La progettazione mobile deve tenere conto di un'attenzione frammentata. L'utente potrebbe trovarsi in situazioni in cui altre attività, come guidare, richiedono la sua attenzione visiva, uditiva o l'uso delle mani, limitando le capacità di interazione con il dispositivo. Anche le condizioni ambientali, come l'illuminazione variabile o i rumori di fondo, possono influenzare l'uso dell'app e rendere più difficile per l'utente concentrarsi sul dispositivo. Di conseguenza, le applicazioni mobili devono essere progettate per minimizzare la complessità visiva e rendere l'interazione rapida e intuitiva.

Un'applicazione mobile si differenzia notevolmente da quelle per desktop, e adattare un'app desktop a un dispositivo mobile richiede scelte specifiche di progettazione. I limiti principali derivano dalle dimensioni dello schermo, dalla memoria disponibile e dalla modalità di interazione utente-dispositivo.

1. *Schermo dello smartphone*: Le dimensioni ridotte degli schermi mobili, tipicamente tra i 5 e i 7 pollici, offrono alta risoluzione ma limitano lo spazio disponibile. Questa caratteristica implica che l'interfaccia non può essere sovraccarica di elementi: solo le informazioni essenziali devono essere visualizzate, garantendo leggibilità e usabilità senza affollamento visivo.
2. *Memoria limitata*: Le risorse di memoria nei dispositivi mobili sono spesso inferiori rispetto ai computer desktop. Pertanto, è importante ridurre al minimo la dimensione dei file e dei dati processati dall'app, eliminando memory leaks e usando formati efficienti per il salvataggio delle informazioni.

Un'altra importante differenza con i sistemi desktop è l'assenza di finestre multiple. In un dispositivo mobile, l'utente accede alle schermate sequenzialmente, e ogni schermata occupa l'intero spazio visibile. Questo limita il multitasking e obbliga a organizzare i contenuti in una sequenza logica e continua. Per applicazioni che, in ambiente desktop, richiedono più finestre aperte contemporaneamente, è necessario riorganizzare le funzioni in una serie di schermate sequenziali o, se possibile, concentrarsi solo su alcune funzionalità essenziali.

Diversamente dai computer desktop, i dispositivi mobili non supportano il multitasking per le app di terze parti. Quando l'utente riceve una chiamata, l'app in uso viene sospesa, richiedendo una progettazione che preveda la possibilità di interrompere l'applicazione senza perdere dati o stato. Quando l'utente riprende l'applicazione, questa dovrebbe essere in grado di riavviarsi rapidamente esattamente dallo stato precedente. Inoltre, se l'app necessita di modifiche nelle impostazioni del sistema, queste possono interrompere l'esperienza d'uso, e quindi si preferisce che il maggior numero possibile di configurazioni sia gestibile all'interno dell'app stessa.

Gli utenti mobili hanno generalmente poco tempo e non sono propensi a leggere istruzioni o sezioni di help approfondite. Per questo motivo, l'interfaccia deve essere intuitiva e basata su controlli standard facilmente riconoscibili. La navigazione dovrebbe essere strutturata con sequenze logiche di schermate che siano immediatamente comprensibili. Inoltre, è fondamentale includere un pulsante "back" in ogni schermata, che permetta all'utente di tornare indietro facilmente senza confusione.

8.1 Stili di app mobili

Le applicazioni mobili si differenziano tra loro non solo per lo scopo, ma anche per lo stile di interazione e la struttura delle informazioni. Ogni tipologia risponde a esigenze specifiche e offre un'esperienza utente diversa, che influenza l'interfaccia e l'organizzazione dei contenuti. Gli stili principali di applicazioni mobili includono: productivity app, utility app e immersive app. Ogni categoria si adatta a differenti esigenze e situazioni d'uso.

8.1.1 Productivity application

Le *productivity app* sono pensate per supportare compiti che richiedono organizzazione e gestione di informazioni dettagliate. Applicazioni come la posta elettronica sono esempi tipici, poiché permettono di svolgere compiti importanti e richiedono un'interazione frequente. La user experience di queste applicazioni è focalizzata sul compito e sulle informazioni necessarie per completarlo, con un'interazione costante e dettagliata.

Un esempio classico di productivity app è Google Photo, che organizza e gestisce foto e video. La struttura dei dati in queste applicazioni è spesso organizzata in modo gerarchico, permettendo agli utenti di navigare tra diversi livelli di dettaglio e di eseguire operazioni specifiche a seconda del livello selezionato. L'interfaccia di queste applicazioni è semplice e pulita, con una schermata dedicata per ogni livello gerarchico, e l'utilizzo di controlli standard, che gli utenti riconoscono facilmente.

Generalmente, in questo tipo di applicazioni, l'utente può innanzitutto organizzare una lista di elementi, aggiungendo o rimuovendo voci a seconda delle necessità. A questo punto, è possibile navigare a livelli di dettaglio successivi: ogni livello presenta informazioni più specifiche sull'elemento selezionato, permettendo all'utente di compiere azioni mirate o approfondimenti direttamente su quel livello.

Le productivity app pongono l'enfasi sulle informazioni e sul compito, lasciando in secondo piano l'ambiente grafico o la ricchezza dell'esperienza visiva. Di solito, gli utenti possono impostare preferenze per ridurre scelte ripetitive e per focalizzarsi solo sulle operazioni essenziali, ottimizzando così il tempo e l'attenzione dedicata al task.

Modi delle productivity app

L'interfaccia di una productivity app presenta generalmente una lista di oggetti organizzata in una struttura ad albero, con una gerarchia che consente di scendere a livelli di dettaglio progressivi. Per esempio, un'app per la gestione della posta elettronica come Gmail mostra inizialmente una lista di conversazioni. Quando si seleziona una conversazione, viene visualizzata una nuova schermata che mostra i messaggi contenuti in quella conversazione, consentendo all'utente di navigare nei vari livelli dell'albero, partendo dalla radice e procedendo attraverso i nodi figli.

All'interno di queste app, gesti come lo swipe permettono di eseguire azioni rapide sulla conversazione stessa, come archiviare o segnare come letto/non letto, senza aprire nuove schermate. Per creare una nuova conversazione, basta toccare il pulsante con l'icona della matita, che apre una nuova schermata di composizione del messaggio.

Un principio chiave delle productivity app è l'utilizzo dell'approccio noun-verb, ossia "oggetto-azione." Con questo metodo, l'utente sceglie prima l'oggetto e poi l'azione da compie-



re su di esso. Ad esempio, in un'app di posta elettronica, è possibile archiviare una conversazione eseguendo uno swipe o eliminarla con un semplice tap su un pulsante specifico. Questo approccio facilita la comprensione e la gestione delle azioni, in quanto l'animazione degli oggetti offre feedback immediato sull'avvenuta esecuzione, eliminando la necessità di alert o messaggi aggiuntivi.

L'approccio noun-verb è particolarmente vantaggioso per gli utenti mobili, poiché riduce gli errori di modo. Infatti, l'attenzione dell'utente è spesso frammentata e potrebbe non ricordare lo stato del sistema o il contesto in cui si trova, aumentando il rischio di errori. L'approccio noun-verb consente di svolgere le operazioni senza creare modi, ovvero situazioni in cui l'interpretazione delle azioni cambia in base allo stato del sistema.

La navigazione nelle productivity app è spesso arricchita da animazioni che facilitano l'esperienza utente. Quando si apre una nuova vista figlia, questa entra da destra e si sovrappone alla vista padre. Per tornare indietro, si può usare il pulsante "back" in alto a sinistra, che provoca uno scorrimento della vista figlia verso destra, rivelando di nuovo la vista padre. Questa struttura, simile a una pila, consente all'utente di mantenere un'idea chiara della gerarchia e della navigazione, con l'ultima schermata inserita che diventa la prima a uscire.

In alcuni casi, come nell'aggiunta di nuovi oggetti, l'interazione adotta l'approccio verb-noun, ovvero "azione-oggetto." Un esempio è la composizione di un nuovo messaggio in un'app di posta elettronica: l'utente preme il pulsante "scrivi" (contrassegnato dall'icona di una matita) e accede a una nuova schermata, dove può iniziare a comporre il messaggio. Questa schermata viene visualizzata in modalità modale, salendo dal basso e coprendo la vista precedente per focalizzare l'attenzione dell'utente sulla composizione del messaggio.

Quando l'utente termina e invia il messaggio, la vista modale scorre verso il basso, rivelando di nuovo la schermata precedente e aggiungendo il nuovo elemento alla lista delle conversazioni con un'animazione, che funge da feedback visivo per confermare l'avvenuto invio.

Molte applicazioni per smartphone iniziano dalla radice dell'albero gerarchico, dove si trova la vista di default. Questa scelta, combinata con l'approccio noun-verb, rende l'applicazione rapida e facile da usare, con una riduzione degli errori. Iniziare dalla radice facilita l'accesso immediato alle funzioni principali e consente all'utente di orientarsi facilmente. Applicazioni come Gmail, WhatsApp, Telegram, Google Drive e BancoPosta utilizzano questo approccio per garantire semplicità e immediatezza.

8.1.2 Utility application

Le *utility app*, o applicazioni di utilità, servono a compiere compiti semplici che richiedono un minimo intervento dell'utente. Il loro scopo principale è fornire informazioni essenziali su un argomento specifico, come nel caso delle applicazioni per le previsioni meteo. In queste app, non c'è struttura gerarchica: l'utente accede direttamente alle informazioni che gli interessano, e l'interazione è ridotta al minimo.

Per esempio, un'app meteo consente all'utente di visualizzare rapidamente le previsioni per varie città. L'interfaccia è tipicamente costituita da una o più liste non gerarchiche, facilitando così la lettura e l'accesso alle informazioni di interesse. Gli utenti possono modificare facilmente le impostazioni, come l'aggiunta di una nuova città o la visualizzazione delle temperature minime, direttamente dall'interno dell'applicazione, rendendo l'esperienza rapida ed efficace.

Nelle utility app, è comune che l'utente apporti frequenti modifiche alla configurazione, come nel caso delle app che monitorano stati o parametri in tempo reale. Queste applicazioni devono quindi essere progettate per semplificare le modifiche delle preferenze e per rispondere in modo dinamico ai cambiamenti dell'utente.

8.1.3 Immersive application

Le *immersive app* sono applicazioni che forniscono un'esperienza immersiva, progettata per trattener l'utente in un ambiente interattivo unico. Queste app, tipiche dei videogiochi o di esperienze virtuali, offrono ambienti ricchi di dettagli e occupano l'intero schermo del dispositivo, permettendo all'utente di esplorare e interagire con il contenuto in modo approfondito. L'attenzione dell'utente è catturata completamente, senza interruzioni o compiti collaterali.

Un esempio di immersive app è l'app IKEA Place, che permette agli utenti di visualizzare come i mobili si integrerebbero nello spazio domestico, utilizzando la realtà aumentata. L'interazione in

queste applicazioni non segue i controlli standard, ma piuttosto introduce comandi unici e personalizzati. Parte dell'esperienza è data dalla scoperta dei comandi stessi e dal loro funzionamento, che coinvolge l'utente in un processo esplorativo.

Queste applicazioni spesso gestiscono grandi quantità di informazioni, ma le presentano in modo adatto al contesto, come dettagli grafici o elementi interattivi che potenziano l'esperienza visiva. L'utente si immerge nell'ambiente dell'app e interagisce con il contenuto come parte dell'esperienza stessa, invece di trattare semplicemente delle informazioni.

8.2 Design patterns

Il concetto di design pattern, o "modello di progettazione," fu introdotto inizialmente in architettura dall'architetto Christopher Alexander.

"Ogni modello descrive un problema che si ripresenta continuamente nel nostro ambiente e, successivamente, illustra il nucleo della soluzione a quel problema, in modo tale che questa soluzione possa essere utilizzata milioni di volte, senza mai applicarla nello stesso modo due volte."

Questa idea fu poi estesa al design delle interfacce utente, con l'obiettivo di fornire soluzioni standardizzate per problemi comuni nel campo dell'interazione uomo-macchina (HCI).

I design patterns sono utili per risolvere problemi di progettazione ricorrenti e forniscono un linguaggio comune per i progettisti. Essi non sono né troppo generali né troppo specifici, il che permette di adattare la soluzione proposta alle particolarità del caso specifico. La loro funzione è quella di facilitare la comunicazione e il confronto tra designer: menzionare un design pattern riconosciuto offre immediatamente a tutti i partecipanti un'idea chiara di cosa si stia discutendo, permettendo di considerare direttamente eventuali alternative.

Ogni interfaccia utente è unica e progettata per rispondere a obiettivi e dati specifici. Tuttavia, ciò non significa che ogni interfaccia debba richiedere agli utenti di imparare nuove convenzioni per utilizzarla. I design patterns aiutano a creare un'interfaccia intuitiva, in cui gli utenti possono riconoscere rapidamente i comandi e le interazioni fondamentali, facilitando l'apprendimento e l'uso dell'applicazione.

Esempi di design patterns comuni nelle interfacce utente includono:

- *Accordion Menu*: un menu che si espande per rivelare le opzioni nascoste al suo interno, fornendo un'interazione intuitiva per navigare tra sezioni di contenuto correlate.
- *Dropdown Menu*: una lista che si espande verso il basso quando selezionata, usata spesso per opzioni selezionabili che devono essere mantenute nascoste fino a quando non servono.
- *Cards*: elementi a forma di "carta" che mostrano contenuti specifici e sono organizzati in griglie o pile, molto usati per contenuti visivi come foto e articoli.
- *Breadcrumbs*: una rappresentazione della posizione corrente dell'utente all'interno della struttura dell'app, utile per facilitare la navigazione all'indietro.
- *Hamburger Menu*: un'icona a tre linee parallele che, se selezionata, apre un menu nascosto con ulteriori opzioni di navigazione.

Nelle applicazioni mobili, soprattutto su Android, esistono design patterns specifici pensati per facilitare la navigazione e migliorare l'esperienza utente. Tra questi si trovano:

- *Toolbar e App Bar*: elementi fissi nella parte superiore dello schermo che includono opzioni come il menu e la barra di ricerca.
- *Tabs*: consentono di navigare tra diverse sezioni di un'app all'interno della stessa schermata, mantenendo un contesto visivo chiaro.
- *Navigation Drawer*: un menu laterale che si apre con uno swipe dal bordo dello schermo, rivelando opzioni di navigazione aggiuntive.
- *Scrolling e Paging*: consentono di esplorare contenuti lunghi in modo fluido, con la possibilità di navigare tra pagine o sezioni con uno swipe.

8.2.1 Dark patterns

Oltre ai design patterns positivi, esistono anche i cosiddetti "dark patterns," termine coniato dal designer Harry Brignull nel 2010 per descrivere quegli schemi di progettazione che inducono deliberatamente gli utenti a prendere decisioni contrarie ai loro interessi. I dark patterns, che Brignull ha esposto in una galleria online e in una campagna di sensibilizzazione, mirano a manipolare gli utenti con tattiche come nascondere le opzioni di annullamento, rendere difficile la disiscrizione o spingere inconsapevolmente verso acquisti non desiderati.

Oggi, alcuni preferiscono sostituire il termine "dark pattern" con "deceptive design" o "design ingannevole" per evitare connotazioni negative legate al colore "dark". L'uso di questo tipo di progettazione è in calo, con un numero crescente di aziende e designer che riconoscono l'importanza di pratiche etiche e trasparenti nell'interazione uomo-macchina.

8.2.2 Attention-capture damaging patterns

L'economia dell'attenzione è un modello di business che si basa sul considerare l'attenzione degli utenti come una valuta. In pratica, le aziende tecnologiche progettano servizi digitali che catturano l'interesse degli utenti, utilizzando il tempo speso sulle piattaforme come una forma di pagamento. Questo modello è il motore dietro colossi tecnologici come Alphabet (Google) e Meta (Facebook, Instagram, WhatsApp), che traggono enormi profitti vendendo l'attenzione e, a volte, i dati degli utenti a inserzionisti.

Tuttavia, questo sistema ha conseguenze significative sul benessere digitale delle persone. Le piattaforme spesso impiegano tecniche che sfruttano le vulnerabilità psicologiche degli utenti, come la gratificazione immediata e il meccanismo delle ricompense variabili, per indurli a trascorrere sempre più tempo online. Il risultato è una perdita di controllo sugli obiettivi personali e una sensazione di pentimento per il tempo perso.

Un sottoinsieme di design pattern, noti come *Attention-Capture Damaging Patterns (ACDPs)*, viene appositamente progettato per massimizzare l'uso continuo delle piattaforme digitali. Questi schemi spesso automatizzano processi per migliorare l'usabilità, ma lo fanno a scapito dell'autonomia decisionale degli utenti. Utilizzano strategie che includono:

- *Notifiche*: progettate per imitare segnali di pericolo, creando un senso di urgenza.
- *Gratificazione immediata*: come i "like" o i commenti, che attivano il rilascio di dopamina nel cervello.
- *Mescolanza di contenuti vecchi e nuovi*: una tattica mutuata dalle macchine da gioco, che mantiene gli utenti in uno stato costante di curiosità.

Alcuni esempi pratici di questi pattern includono:

1. *Raccomandazioni basate sui gusti dell'utente*: Le piattaforme suggeriscono contenuti che potrebbero piacere all'utente, ma con l'obiettivo di tenerlo impegnato più a lungo. Sebbene utili in alcune circostanze, queste raccomandazioni possono diventare una trappola, portando a un uso non intenzionale della piattaforma.
2. *Autoplay infinito*: Al termine di un video, un altro inizia automaticamente, impedendo agli utenti di fermarsi e riflettere. Questo meccanismo sfrutta la pigrizia decisionale e il principio della ricompensa variabile.
3. *Pull-to-refresh*: Simile al gesto di tirare la leva di una slot machine, scorrere verso il basso su uno smartphone ricarica la pagina e potrebbe rivelare nuovi contenuti, incentivando un comportamento compulsivo.
4. *Scrolling infinito*: Scorrendo verso l'alto su una pagina, nuovi contenuti vengono caricati automaticamente. Questo riduce l'attrito nella navigazione ma incoraggia sessioni di utilizzo senza fine.

8.3 Interfacce Android

La progettazione delle interfacce per le applicazioni Android segue una serie di criteri fondamentali per migliorare l'esperienza utente. Questi principi si basano su una comprensione profonda delle capacità cognitive degli utenti, delle loro aspettative e dei modelli di interazione già conosciuti.

8.3.1 Criteri

Un design efficace utilizza gli elementi che gli utenti già conoscono. Questo principio si declina in tre aspetti fondamentali:

1. *Widget standard.* I widget sono gli elementi grafici predefiniti che gli utenti incontrano comunemente, come pulsanti, slider e menu a tendina. Utilizzare questi elementi riduce il tempo necessario per imparare a usare l'applicazione, poiché gli utenti riconoscono immediatamente le loro funzioni.
2. *Gesture standard.* Le gesture, come lo swipe o il pinch-to-zoom, sono comportamenti acquisiti nella navigazione digitale. Un'applicazione che adotta queste interazioni standard migliora l'accessibilità e la soddisfazione dell'utente.
3. *Metafore riconoscibili.* L'uso di metafore visive come un'icona del cestino per eliminare un file aiuta l'utente a comprendere rapidamente la funzione di un elemento. È importante scegliere immagini che siano culturalmente universali e intuitive.

La *discoverability* si riferisce alla capacità degli utenti di individuare facilmente le funzioni principali di un'interfaccia. Un errore comune è nascondere i controlli, rendendo l'interazione con l'applicazione più difficile.

Ad esempio, Material Design consiglia di evitare che i menu siano completamente nascosti dietro icone, se non necessario. Fornire un'indicazione visiva chiara aumenta la probabilità che l'utente esplori e utilizzi le funzionalità.

Una buona interfaccia deve rispondere rapidamente e chiaramente alle azioni dell'utente, rendendo l'interazione più naturale. Questo criterio comprende due aspetti principali:

1. *Manipolazione diretta.* Gli utenti dovrebbero essere in grado di interagire direttamente con gli elementi della schermata, come trascinare un oggetto o ridimensionarlo.
2. *Feedback immediato.* Ogni azione deve essere seguita da una reazione visibile. Ad esempio, quando si preme un pulsante, un cambiamento di colore o un'animazione può confermare che l'azione è stata registrata.

L'interfaccia dovrebbe ridurre il carico cognitivo richiedendo agli utenti di selezionare, piuttosto che ricordare. Offrire menu a scelta o suggerimenti visivi facilita la navigazione e migliora l'efficienza.

Un esempio tipico è l'uso di menu a discesa per consentire la scelta di opzioni predefinite, invece di costringere l'utente a digitare manualmente.

Gli utenti devono sentirsi al comando durante l'interazione con un'app. Ciò si traduce in funzionalità come:

- *Orientamento chiaro.* Mostrare all'utente dove si trova nell'applicazione attraverso barre di navigazione o breadcrumbs.
- *Navigazione inversa.* Funzioni come il pulsante "Indietro" permettono di annullare facilmente un'azione o tornare a una schermata precedente.
- *Progress bar e undo.* Una barra di avanzamento fornisce un'indicazione chiara del progresso dell'utente, mentre l'opzione "Undo" offre un modo semplice per correggere errori.

Un'app efficace apprende e si adatta alle preferenze dell'utente.

- *Suggerimenti personalizzati.* L'applicazione può proporre impostazioni basate sull'uso passato.
- *Personalizzazione attiva.* Offrire agli utenti la possibilità di modificare impostazioni come temi, layout o notifiche.
- *Default sensati.* Impostazioni predefinite ben progettate riducono la necessità di personalizzazioni immediate.

Una buona interfaccia è coerente sia visivamente che comportamentalmente. Questo principio include:

- *Stesse gesture, stessa azione.* Un'azione deve sempre avere lo stesso risultato, indipendentemente dal contesto.

- *Evitare i modi.* Un'interfaccia che cambia funzione di un elemento in base al contesto può confondere.
- *Coerenza tra widget e funzioni.* Un widget, come un bottone o un campo di testo, deve sempre comportarsi in modo prevedibile.
- *Temi uniformi.* L'aspetto grafico deve essere armonizzato per creare un'identità visiva forte.

8.3.2 Material design

Material Design è un linguaggio visivo sviluppato da Google per garantire un'esperienza coerente, intuitiva e moderna su tutte le piattaforme e i dispositivi Android.

Material Design si ispira alle proprietà fisiche dei materiali del mondo reale per creare un'interfaccia digitale che sembri tattile e naturale. Ogni elemento, come pulsanti e schede, simula comportamenti fisici, come la luce e le ombre, per fornire una percezione visiva di profondità e gerarchia.

Lists

Le liste sono un componente centrale del Material Design, utilizzate per organizzare informazioni in un formato verticale continuo e per mostrare contenuti multipli in modo ordinato. Possono includere testo e immagini, con l'obiettivo di aiutare gli utenti a trovare e agire su un elemento specifico. Le liste si classificano in tre dimensioni: a una linea, due linee o tre linee.

Le liste sono progettate per essere continue e coerenti, in modo che l'utente possa scorrere agevolmente senza interruzioni visive.

Cards

Le cards sono un altro elemento distintivo del Material Design. Sono elementi versatili che rappresentano un contenuto organizzato intorno a un singolo argomento. Possono contenere testi, immagini, pulsanti e icone, offrendo un punto di accesso a dettagli più approfonditi o ulteriori navigazioni. La loro flessibilità permette l'uso in vari layout, come griglie, elenchi verticali o caroselli.

Buttons

I bottoni sono componenti essenziali che comunicano azioni disponibili. In Material Design, i bottoni hanno angoli completamente arrotondati e dimensioni tali da contenere comodamente il testo dell'etichetta. Possono includere icone opzionali, posizionate a sinistra rispetto al testo. È importante usarli con moderazione per evitare di compromettere la gerarchia visiva.

Floating Action Buttons (FAB)

Il Floating Action Button è progettato per evidenziare l'azione più importante in una schermata, come "Aggiungi", "Condividi" o "Avvia". Questo bottone fluttuante è persistente durante lo scorrimento e si trova solitamente in un contenitore arrotondato sopra la barra di navigazione.

Alerts

Gli avvisi sono messaggi generati dal sistema per informare l'utente di situazioni critiche o che richiedono una decisione immediata. Possono essere utilizzati per segnalare errori, chiedere conferme o fornire notifiche importanti. Gli avvisi devono essere chiari, immediatamente riconoscibili e mai troppo invasivi.

Bottom Sheets

I bottom sheets sono pannelli che si sollevano dal bordo inferiore dello schermo, mostrando contenuti o azioni supplementari. Possono essere di due tipi: standard, utilizzati per integrare i contenuti principali (come un lettore audio), o modali, che offrono un'alternativa ai menu inline, spesso coprendo l'intero schermo.

Questo componente è particolarmente utile per mantenere l'interfaccia pulita, mostrando solo le opzioni rilevanti quando necessario.

Side Sheets

I side sheets mostrano contenuti secondari ancorati ai lati dello schermo. Sono utili per azioni come filtri o opzioni che influenzano i contenuti principali. Possono essere standard, ideali per schermi ampi, o modali, pensati per dispositivi con spazi più ristretti.

Tabs

Le tabs organizzano i contenuti in diverse categorie, offrendo un metodo di navigazione intuitivo tra sezioni correlate. Ogni tab include un'etichetta e, opzionalmente, un'icona. L'utente può passare da una tab all'altra scorrendo lateralmente o cliccando direttamente sull'etichetta.

Barre dell'applicazione

Material Design distingue tra due tipi di barre applicative: la top app bar e la bottom app bar.

- La *top app bar* include il titolo della schermata corrente e opzioni di navigazione come il menu "hamburger" o una freccia per tornare indietro. Scorrendo verso il basso, questa barra scompare per massimizzare lo spazio visibile.
- La *bottom app bar*, invece, ospita fino a quattro pulsanti di azione e può integrare un Floating Action Button. È adatta a schermate con molte opzioni di azione.

Navigation Bar

La barra di navigazione (precedentemente chiamata "bottom navigation") consente l'accesso rapido a 3-5 destinazioni di pari importanza. Ogni destinazione è rappresentata da un'icona e, opzionalmente, un'etichetta. Questa barra è ideale per le app che richiedono un accesso costante a più sezioni principali, come "Home", "Profilo" e "Impostazioni".

Transizioni

Le transizioni definiscono il modo in cui i contenuti si spostano e interagiscono durante la navigazione. Possono essere:

- Avanti e indietro, per navigare tra livelli gerarchici.
- Lateralmente, per esplorare contenuti paralleli, come il passaggio tra tabs.

Sistema cromatico

Material Design 3 (M3) introduce un sistema cromatico sofisticato per garantire coerenza, accessibilità e personalizzazione nelle interfacce utente. Questo sistema consente di creare temi statici partendo da un colore di riferimento, utilizzando algoritmi per generare palette cromatiche coerenti e ben bilanciate. Un aspetto interessante è la sua versatilità: i temi creati con M3 sono applicabili anche su piattaforme diverse da Android, come iOS.

Il sistema cromatico è centrale nella progettazione visiva:

- Promuove una coerenza visiva che rafforza l'identità del brand.
- Integra principi di accessibilità, garantendo contrasti ottimali per diverse situazioni visive.
- Offre opzioni di personalizzazione per adattarsi a ogni esigenza di design.

Il modello cromatico di M3 si basa su tre componenti fondamentali:

1. *Ruoli dei colori (color roles)*: Ogni elemento visivo è associato a un ruolo specifico, come "primary" per i pulsanti principali o "background" per lo sfondo generale.
2. *Assegnazione dinamica*: Gli elementi visivi ricevono un colore sulla base del loro ruolo.
3. *Algoritmo della palette*: A partire da un colore di riferimento, noto come seed color, il sistema genera tonalità e livelli di luminosità, adattandoli ai vari contesti.

I ruoli non indicano un colore preciso, ma descrivono il loro utilizzo. Ad esempio, un pulsante etichettato come "primary" utilizzerà un colore specifico in base alla modalità chiara o scura selezionata.

Ecco alcuni ruoli comuni:

- *Primary*: Colore degli elementi interattivi principali.
- *Secondary*: Accenti visivi e complementi.
- *Tertiary*: Variazioni stilistiche opzionali.
- *Background*: Colore dello sfondo generale.
- *Surface*: Superfici come schede o finestre.
- *Error*: Colore per messaggi di errore.
- *Outline*: Bordature o contorni di elementi.
- *On-Roles*: Testi o icone sovrapposti a uno sfondo, come "on-primary" o "on-background".

Material Design 3 utilizza un algoritmo avanzato noto come Material Color Utilities (MCU). Questo sistema:

1. Richiede un seed color come input.
2. Genera 5 colori complementari chiave (key colors).
3. Crea una palette di tonalità per ciascun key color, con 13 livelli di luminosità.
4. Assegna i colori generati ai ruoli principali dell'interfaccia.

La distribuzione dei colori avviene in base a:

- La modalità del tema (chiara o scura).
- Le regole di accessibilità, per garantire che ogni elemento sia ben visibile.

Material Design 3 semplifica il processo di creazione dei temi grazie a strumenti integrati, come il plug-in Material Theme Builder per Figma. Questo strumento consente di:

1. Inserire un seed color.
2. Generare automaticamente una palette cromatica.
3. Assegnare i colori generati ai color roles.
4. Applicare i temi a componenti predefiniti.

Utilizzando il M3 Design Kit in Figma, si possono creare prototipi di interfaccia che rispettano i ruoli cromatici. Il processo tipico prevede:

1. Selezione del colore rappresentativo del brand come seed.
2. Generazione automatica della palette e aggiornamento del tema.
3. Applicazione del tema ai componenti dell'interfaccia tramite la funzione "swap".

8.4 Interfacce IOS

Progettare interfacce per iOS richiede il rispetto di criteri specifici che garantiscano un'esperienza utente fluida e coerente. Vedremo i principi fondamentali come l'autenticazione, la navigazione, il branding e altri elementi chiave che rendono un'app intuitiva e in linea con le aspettative degli utenti iOS.

8.4.1 Criteri

Un aspetto essenziale della progettazione di applicazioni è l'implementazione del *processo di autenticazione*. Per migliorare l'esperienza utente, si consiglia di posticipare il più possibile il momento in cui l'utente deve effettuare il login, preferibilmente dopo aver dimostrato i vantaggi di tale operazione. Minimizzare i dati da inserire è altrettanto importante: l'utilizzo di metodi come l'accesso tramite social network o il riconoscimento biometrico può ridurre significativamente gli attriti.

L'*inserimento di dati* manuali può essere un compito noioso e incline agli errori. È preferibile utilizzare selettori piuttosto che richiedere all'utente di digitare informazioni, ricorrendo a dati già presenti nel sistema (come contatti o indirizzi). Gli sviluppatori possono inoltre sfruttare impostazioni predefinite, validazioni dinamiche e suggerimenti visivi per semplificare l'esperienza.

Le *gesture*, ovvero i gesti tattili, sono un elemento distintivo delle interfacce mobili. Le applicazioni iOS dovrebbero adottare gesture standardizzate, evitando di associare comportamenti non convenzionali a queste ultime.

Le *viste modali* sono strumenti utili per focalizzare l'attenzione dell'utente su compiti specifici. Queste viste possono occupare lo schermo parzialmente o completamente e includono sempre controlli chiari per completare o annullare l'operazione. È fondamentale che i compiti modali siano pochi, semplici e brevi.

Un'app ben progettata mantiene chiaro il percorso di *navigazione* per l'utente. Questo obiettivo può essere raggiunto utilizzando componenti standard come barre di navigazione, barre delle tab e controlli di paginazione. Inoltre, l'adozione di gesture familiari, come il gesto di scorrimento per tornare indietro, rafforza ulteriormente l'intuitività dell'interfaccia.

Il *branding* deve essere discreto e ben integrato nella grafica dell'app, evitando di risultare invasivo. Gli elementi visivi, come i loghi, dovrebbero essere coerenti in tutta l'app ma non collocati nella barra di navigazione, dove potrebbero distrarre l'utente.

La *schermata di avvio* di un'applicazione iOS dovrebbe ripristinare il contesto dell'utente e permettere di riprendere eventuali attività interrotte. Questo approccio riduce la frustrazione e migliora la continuità dell'esperienza. La schermata di avvio deve essere identica all'interfaccia principale, senza contenere testi o elementi dipendenti dalla localizzazione, al fine di garantire coerenza e rapidità.

8.4.2 Strumenti

Le principali funzionalità offerte dal sistema operativo iOS sono Haptic Touch, le notifiche, i widget della schermata Home e Siri.

- *Haptic Touch*. Haptic Touch è una tecnologia che combina il gesto di una pressione prolungata sullo schermo con un feedback tattile, simulando la sensazione di premere un tasto fisico. Questa interazione può attivare un menu contestuale, mostrare contenuti aggiuntivi o fornire un'anteprima.

Haptic Touch supporta diverse interazioni, come il "peek" e il "pop". La pressione iniziale, seguita da una vibrazione, consente di sbirciare un contenuto (peek) visualizzando una preview. Mentre una pressione prolungata permette di accedervi completamente (pop). Se al peek segue uno swipe verso l'alto viene mostrato un menù di action buttons per l'item selezionato.

- *Notifiche*. Le notifiche rappresentano uno strumento fondamentale per comunicare informazioni importanti all'utente. iOS supporta notifiche locali e push, che possono essere configurate per includere un titolo, un testo descrittivo e un suono. È interessante notare che le notifiche non mostrano il nome o il logo dell'app, concentrandosi esclusivamente sul contenuto rilevante.

Le notifiche possono apparire sotto forma di banner, avvisi o direttamente sulla schermata di blocco. L'interazione con queste notifiche è altrettanto versatile: un semplice tap permette di aprirle, mentre uno swipe verso l'alto le cancella. Inoltre, con una pressione prolungata (Haptic Touch) o uno swipe verso il basso, l'utente può visualizzare una versione espansa che mostra maggiori dettagli o opzioni di azione.

- *Home Screen Widgets*. I widget della schermata Home sono un modo pratico per offrire informazioni essenziali o funzionalità rapide direttamente sulla schermata principale o nella vista

“Oggi”. Ogni app può includere più widget, progettati per essere efficienti e consumare meno risorse grazie alla gestione intelligente della cache. I widget possono contenere bottoni, testo, immagini e altre componenti interattive.

- *Siri*. L'integrazione con Siri permette alle applicazioni di iOS di offrire esperienze vocali naturali e intuitive. Utilizzando App Intents, gli sviluppatori possono definire i task supportati dall'app, convalidare gli input dell'utente, eseguire i compiti richiesti e fornire un feedback. Siri Shortcuts consente inoltre di semplificare le interazioni, proponendo opzioni chiare e immediate.

8.4.3 Human Interface Guidelines

La progettazione delle interfacce utente per applicazioni iOS è regolata da una serie di linee guida chiamate Human Interface Guidelines (HIG). Queste direttive hanno lo scopo di creare esperienze intuitive e coerenti per gli utenti, promuovendo pratiche che rendano le interazioni con le app fluide ed efficaci. Vedremo alcuni degli elementi chiave utilizzati nelle interfacce grafiche, come barre, viste e controlli. Questi componenti non solo migliorano la navigazione e l'usabilità, ma offrono anche agli sviluppatori una base solida per costruire esperienze utente di alta qualità.

Bar

- La status bar è un elemento essenziale dell'interfaccia iOS che fornisce informazioni sullo stato corrente del dispositivo, come la connessione di rete, l'orario e il livello della batteria. Questo componente viene generato automaticamente dal sistema operativo e dovrebbe essere sempre visibile. In caso di operazioni lunghe, come il caricamento di dati dalla rete, può essere utilizzato un indicatore di progresso indeterminato per comunicare all'utente che l'operazione è in corso.
- La navigation bar è fondamentale per la navigazione all'interno di un'applicazione. Essa include un pulsante "Indietro", un titolo che rappresenta la schermata corrente e, talvolta, un controllo segmentato o un pulsante aggiuntivo. Tuttavia, non è necessaria se la vista corrente non permette ulteriori navigazioni. iOS consente anche di utilizzare titoli grandi per migliorare la gerarchia visiva, rendendo più chiaro all'utente in quale sezione dell'app si trova.
- La toolbar contiene bottoni o etichette che permettono di eseguire azioni rilevanti nella vista corrente. Deve essere compatta e chiara, con un massimo di cinque icone selezionate da un set predefinito e prive di didascalie, oppure tre etichette di testo dinamiche.
- La tab bar è utilizzata per navigare tra diverse sezioni di un'app. Ogni tab può includere un'icona personalizzata e una breve etichetta. Sebbene sia possibile avere fino a cinque tab, in caso di sezioni aggiuntive viene mostrata una voce "Altro". La tab bar è sempre visibile e può includere badge dinamici per indicare notifiche o aggiornamenti.

È importante utilizzare il numero minimo necessario di tab per facilitare la navigazione, mantenendoli visibili anche quando il loro contenuto non è disponibile. Per ogni tab, è consigliabile utilizzare termini sintetici ed evitare etichette generiche come "Home", che potrebbero risultare ambigue.

- Le sidebars offrono un metodo pratico per navigare in un'applicazione, fornendo accesso rapido alle principali aree o collezioni di contenuti. Gli utenti si aspettano di poter mostrare o nascondere la sidebar tramite il gesto integrato di swipe dal bordo dello schermo. Tuttavia, si consiglia di limitare la gerarchia a non più di due livelli, per evitare complessità. Nelle applicazioni iOS, è preferibile considerare l'uso di una tab bar al posto di una sidebar, poiché risulta più intuitiva e conforme alle linee guida di design.

Views

- Una collezione rappresenta un insieme ordinato di contenuti disposti in un layout altamente visivo, che può essere una griglia o una fila orizzontale. Questo approccio è ideale per contenuti visivi come immagini, mentre per il testo è preferibile utilizzare una tabella. Gli utenti possono interagire tramite tap per selezionare un elemento, tenere premuto per modificare o effettuare uno swipe per scorrere.

- Le liste e le tabelle sono strutture utilizzate per presentare dati in colonne e righe, rendendo i contenuti facilmente leggibili e scansionabili.

Le tabelle sono uno strumento versatile per mostrare contenuti organizzati. Sono strutturate in una singola colonna e possono essere scorrevoli verticalmente. Gli elementi nelle tabelle possono essere raggruppati in categorie (viste grouped) o elencati in modo semplice (viste plain). Queste tabelle devono essere aggiornate frequentemente per riflettere dati in tempo reale.

Le righe delle tabelle possono variare in base al formato dei dati. Possono includere titoli, sottotitoli o valori correlati disposti su due colonne (value1 e value2), rendendo il contenuto facilmente leggibile e organizzato.

- I pulsanti sono progettati per attivare azioni immediate. È fondamentale che siano visivamente distinguibili dagli altri elementi dell'interfaccia.

Ogni pulsante dovrebbe avere un'etichetta di testo, un'icona o una combinazione di entrambi. Lo spazio intorno al pulsante deve essere sufficiente per evidenziarlo visivamente. Si consiglia di limitare il numero di pulsanti visivamente prominenti a uno o due per schermata per evitare confusione.

I Floating Action Buttons (FAB) sono widget fondamentali nell'ecosistema Android, ma non sono raccomandati per le applicazioni iOS. Gli utenti iOS potrebbero non comprendere come interagire con questo elemento, poiché non è parte delle loro convenzioni di utilizzo. Invece, è preferibile posizionare un pulsante per le azioni principali nella parte superiore destra della navigation bar, rendendo l'azione più intuitiva e coerente con le aspettative degli utenti.

- I segmented controls sono utili per presentare un numero limitato di opzioni strettamente correlate, che gli utenti possono confrontare rapidamente. Possono essere utilizzati per alternare viste o modalità all'interno della stessa schermata e possono contenere testo, icone o entrambi. Il numero ideale di opzioni è compreso tra 2 e 5. È consigliabile implementare animazioni fluide per evidenziare le modifiche di stato, fornendo un feedback immediato agli utenti.
- Gli alert sono notifiche asincrone utilizzate solo per comunicare informazioni critiche. Questi elementi interrompono l'attività corrente e richiedono all'utente di prendere una decisione. Gli alert devono essere progettati con titoli e testi brevi, con uno o più pulsanti per le azioni.

Gli alerts sono viste modali progettate per fornire informazioni importanti e non attese che l'utente deve affrontare immediatamente. Sono ideali per segnalare problemi, avvisare dell'eliminazione di dati o confermare un acquisto. Non dovrebbero essere utilizzati per semplici comunicazioni informative o azioni comuni che possono essere annullate, anche quando queste sono distruttive.

- Gli action sheets vengono visualizzati su richiesta dell'utente per presentare opzioni relative all'azione corrente. Hanno un layout chiaro, senza contenuti scrollabili, e includono sempre un pulsante per annullare l'operazione.

Gli action sheets rappresentano un'alternativa efficace agli alert o ai menu per offrire scelte relative a un'azione intenzionale da parte dell'utente. Un esempio comune è l'opzione per annullare la scrittura di un'email. L'action sheet deve includere un pulsante per annullare l'operazione, posizionato in fondo, oltre a opzioni aggiuntive correlate all'azione.

- Le activity views sono progettate per eseguire task specifici nel contesto corrente, come la condivisione di un documento o l'invio di un file. Questi elementi sono forniti di default ma possono essere personalizzati. Gli sviluppatori possono utilizzare pulsanti di azione per attivare queste viste, garantendo un'interazione semplice e diretta.

Controls

I controlli permettono agli utenti di interagire con l'applicazione, inserendo dati o selezionando opzioni. Un esempio importante sono i text fields, che devono supportare tastiere appropriate al tipo di input richiesto e funzioni come l'input predittivo per migliorare l'efficienza.

Link utili

- [Incidental interaction](#)
- [Pen and Touch Computing Center](#)
- [How to Conduct User Observations](#)
- [Stop Asking Users What They Want](#)
- [Likert scale](#)
- [Likert Scale Questionnaire: Examples & Analysis](#)
- [Contextual Inquiry: Inspire Design by Observing and Interviewing Users in Their Context](#)
- [Need Finding Tools](#)
- [Task Analysis](#)
- [Creating Personas](#)
- [A Closer Look At Personas: What They Are And How They Work](#)
- [Prototyping](#)
- [Creating Flows With Paper Prototypes](#)
- [How to Test a Paper Prototype](#)
- [Scott Klemmer, Storyboards, Paper Prototypes, and Mockups](#)
- [Mobile Application Design: Paper Prototype Video](#)
- [Affordances and signifiers](#)
- [UI Design Patterns](#)
- [Dark Patterns](#)
- [Material Design 3](#)
- [Mobbin per Android](#)
- [Material Design 3 - Color](#)
- [Human Interface Guidelines](#)
- [Mobbin per iOS](#)
- [Hierarchical navigation in iOS](#)